

Отчет по лабораторной работе №8

Егор Витальевич Кузьмин

Содержание

1	Цель работы	6
2	Используемые файлы и структура работы	7
3	Проверка окружения Julia	9
4	Source-модель <code>sir_model.jl</code>	10
5	Базовый запуск DES SIR-модели	25
5.1	Скрипт <code>sir_des.jl</code>	25
5.2	Выполнение базового скрипта	30
5.3	Динамика SIR-модели	31
5.4	Динамика инфицированных	32
5.5	Финальное состояние	33
6	Literate-версия первого скрипта	34
7	Параметризованная версия первого скрипта	42
7.1	Скрипт <code>sir_des_params.jl</code>	42
7.2	Влияние β	54
7.3	Влияние среднего числа контактов	55
7.4	Влияние γ	56
7.5	Итоговый размер эпидемии по сценариям	57
8	Аналитический скрипт	58
8.1	Скрипт <code>sir_des_analysis.jl</code>	58
8.2	Сравнение DES и ODE	76
8.3	Сценарий вакцинации	77
8.4	Фиксированная длительность болезни	78
8.5	Сравнение пиков инфекции	79
8.6	Сравнение итогового размера эпидемии	80
8.7	Производительность	81
9	Literate-версия аналитического скрипта	82
10	Параметризованная аналитическая версия	104
10.1	Скрипт <code>sir_des_analysis_params.jl</code>	104

10.2	Параметризованный сценарий вакцинации	131
10.3	Фиксированная длительность болезни	132
10.4	Демографический сценарий	133
10.5	SEIR-сценарий	134
10.6	Производительность в параметризованной версии	135
11	Выполнение дополнительных заданий	136
12	Итоговый вывод	137
13	Список литературы	138

Список иллюстраций

3.1	Проверка установленных пакетов Julia	9
5.1	Выполнение и результаты базового скрипта	30
5.2	SIR DES dynamics	31
5.3	SIR DES infected dynamics	32
5.4	SIR DES final state	33
6.1	Создание и notebook literate-версии первого скрипта	41
7.1	Создание параметризованной версии и notebook первого скрипта	53
7.2	Результаты параметризованной версии первого скрипта	53
7.3	SIR DES: effect of beta on infection peak	54
7.4	SIR DES: effect of contacts on infection peak	55
7.5	SIR DES: effect of gamma on infection peak	56
7.6	SIR DES: final epidemic size by scenario	57
8.1	Выполнение и результаты аналитического скрипта	75
8.2	SIR: DES and ODE infected comparison	76
8.3	SIR DES: base and vaccination scenarios	77
8.4	SIR DES: stochastic and fixed recovery	78
8.5	SIR: infection peak comparison	79
8.6	SIR: final epidemic size comparison	80
8.7	SIR DES: performance by population size	81
9.1	Создание и notebook literate-версии второго скрипта	103
10.1	Создание и notebook параметризованной версии второго скрипта	130
10.2	Результаты параметризованной версии второго скрипта	131
10.3	SIR DES: vaccination parameter scan	131
10.4	SIR DES: fixed recovery duration scan	132
10.5	SIR DES: demography parameter scan	133
10.6	SEIR DES: sigma parameter scan	134
10.7	SIR DES: performance parameter scan	135

Список таблиц

1 Цель работы

Цель лабораторной работы — реализовать и исследовать SIR-модель в дискретно-событийном подходе. В ходе работы была создана базовая source-модель, выполнен запуск основной DES-симуляции, подготовлены `literate`- и параметризованные версии скриптов, а также проведён расширенный анализ дополнительных сценариев.

В работе рассматриваются следующие группы агентов:

- S — восприимчивые агенты;
- I — инфицированные агенты;
- R — выздоровевшие или удалённые из процесса распространения инфекции.

Дискретно-событийный подход отличается от непрерывного тем, что состояние системы изменяется не на каждом малом шаге времени, а только в моменты наступления событий. В данной реализации такими событиями являются заражение, выздоровление, вакцинация, а в дополнительных сценариях также демографические события и переходы SEIR-модели.

2 Используемые файлы и структура работы

В лабораторной работе были подготовлены следующие основные файлы:

Файл	Назначение
src/sir_model.jl	source-модуль с базовой DES SIR-моделью и вспомогательными функциями
scripts/sir_des.jl	обычный базовый запуск DES SIR-модели
scripts/sir_des_literate.jl	literate-версия базового запуска
scripts/sir_des_params.jl	параметризованная версия базового запуска
scripts/sir_des_analysis.jl	второй аналитический скрипт со сравнением и дополнительными сценариями
scripts/sir_des_analysis_literate.jl	literate-версия аналитического скрипта
scripts/sir_des_analysis_params.jl	параметризованная аналитическая версия
scripts/tangle.jl	вспомогательный файл для генерации производных файлов из literate-скриптов

На первом этапе была реализована базовая модель, затем выполнен запуск `sir_des.jl`. После этого были подготовлены `literate`- и параметризованные версии первого скрипта. На втором этапе был создан аналитический скрипт `sir_des_analysis.jl`, в котором выполнено сравнение с детерминированной SIR-моделью, рассмотрена вакцинация, фиксированная длительность болезни и оценена производительность. Затем для него также были подготовлены `literate`- и параметризованные версии.

3 Проверка окружения Julia

Перед выполнением моделирования была проверена установка необходимых пакетов Julia. На скриншоте показано, что в окружении проекта присутствуют библиотеки, необходимые для работы: DataFrames, CSV, Plots, DrWatson, DifferentialEquations, Distributions, Random, Literate и другие.

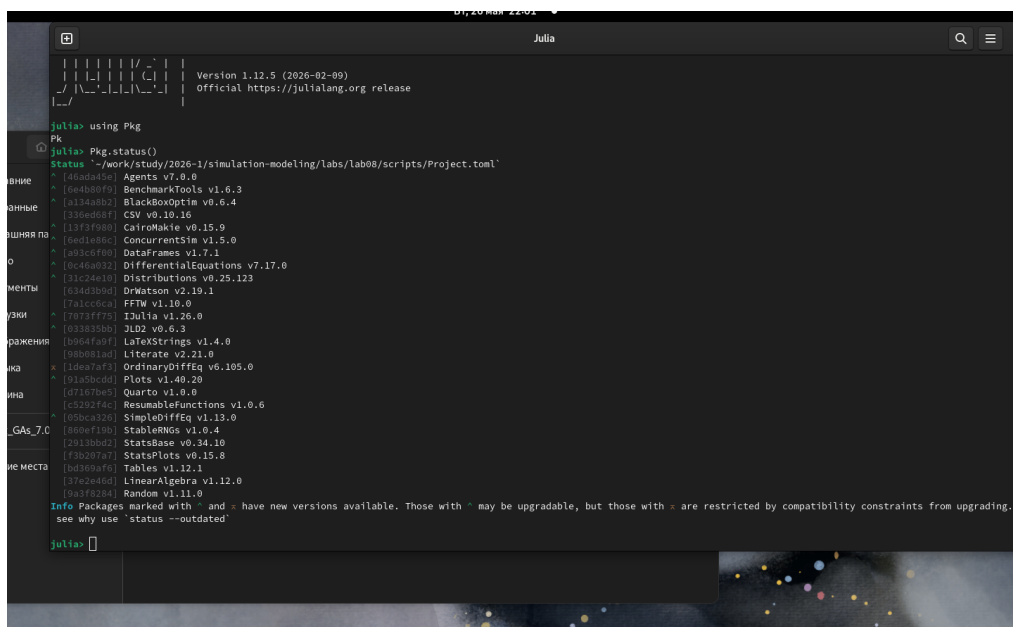


Рисунок 3.1: Проверка установленных пакетов Julia

Наличие этих пакетов позволяет запускать дискретно-событийную модель, сохранять результаты в CSV, строить графики, решать детерминированную систему ОДУ и формировать literate-версии скриптов.

4 Source-модель `sir_model.jl`

Основная логика модели вынесена в файл `src/sir_model.jl`. Такой подход позволяет отделить вычислительную модель от скриптов запуска и анализа. В модуле определены структуры параметров, агента и результата симуляции, а также функции для запуска базовой DES SIR-модели, сценария вакцинации и формирования итоговых таблиц.

```
1 module SIRModel
2
3 using Random
4 using Distributions
5 using DataFrames
6
7 export SIRParameters,
8         SIRPerson,
9         SIRSimulationResult,
10        sir_basic_reproduction_number,
11        initialize_population,
12        simulate_sir_des,
13        simulate_sir_des_vaccination,
14        result_dataframe,
15        events_dataframe,
16        summary_dataframe
```

```

17
18 struct SIRParameters
19     S0::Int
20     I0::Int
21     R0::Int
22     beta::Float64
23     c::Float64
24     gamma::Float64
25     tmax::Float64
26     seed::Int
27 end
28
29 mutable struct SIRPerson
30     id::Int
31     status::Symbol
32 end
33
34 struct SIRSimulationResult
35     params::SIRParameters
36     time::Vector{Float64}
37     S::Vector{Int}
38     I::Vector{Int}
39     R::Vector{Int}
40     events::DataFrame
41 end
42
43 # Базовое репродуктивное число.
44 function sir_basic_reproduction_number(params::SIRParameters)

```

```

45     return params.beta * params.c / params.gamma
46 end
47
48 # Создание начальной популяции.
49 function initialize_population(params::SIRParameters)
50     population = SIRPerson[]
51     id = 1
52
53     for _ in 1:params.S0
54         push!(population, SIRPerson(id, :S))
55         id += 1
56     end
57
58     for _ in 1:params.I0
59         push!(population, SIRPerson(id, :I))
60         id += 1
61     end
62
63     for _ in 1:params.R0
64         push!(population, SIRPerson(id, :R))
65         id += 1
66     end
67
68     return population
69 end
70
71 # Подсчёт количества агентов в состояниях S, I и R.
72 function count_statuses(population::Vector{SIRPerson})

```

```

73     S = count(person -> person.status == :S, population)
74     I = count(person -> person.status == :I, population)
75     R = count(person -> person.status == :R, population)
76
77     return S, I, R
78 end
79
80 # Выбор случайного агента с заданным статусом.
81 function random_person_with_status(
82     rng::AbstractRNG,
83     population::Vector{SIRPerson},
84     status::Symbol,
85 )
86     indexes = findall(person -> person.status == status, population)
87
88     if isempty(indexes)
89         return nothing
90     end
91
92     return population[rand(rng, indexes)]
93 end
94
95 # Добавление текущего состояния во временные ряды.
96 function push_state!(
97     time::Vector{Float64},
98     S_values::Vector{Int},
99     I_values::Vector{Int},
100    R_values::Vector{Int},

```

```

101     t::Float64,
102     population::Vector{SIRPerson},
103 )
104     S, I, R = count_statuses(population)
105
106     push!(time, t)
107     push!(S_values, S)
108     push!(I_values, I)
109     push!(R_values, R)
110
111     return nothing
112 end
113
114 # Добавление события в таблицу событий.
115 function add_event!(
116     events::DataFrame,
117     t::Float64,
118     event::Symbol,
119     person_id::Int,
120     population::Vector{SIRPerson},
121 )
122     S, I, R = count_statuses(population)
123
124     push!(
125         events,
126         (
127             t = t,
128             event = event,

```

```

129         person_id = person_id,
130         S = S,
131         I = I,
132         R = R,
133     ),
134 )
135
136     return nothing
137 end
138
139 # Базовая дискретно-событийная SIR-модель.
140 function simulate_sir_des(params::SIRParameters)
141     rng = MersenneTwister(params.seed)
142     population = initialize_population(params)
143
144     time = Float64[]
145     S_values = Int[]
146     I_values = Int[]
147     R_values = Int[]
148
149     events = DataFrame(
150         t = Float64[],
151         event = Symbol[],
152         person_id = Int[],
153         S = Int[],
154         I = Int[],
155         R = Int[],
156     )

```

```

157
158     t = 0.0
159     N = params.S0 + params.I0 + params.R0
160
161     push_state!(time, S_values, I_values, R_values, t, population)
162
163     while t < params.tmax
164         S, I, _ = count_statuses(population)
165
166         if I == 0
167             break
168         end
169
170         infection_rate = params.beta * params.c * S * I / N
171         recovery_rate = params.gamma * I
172         total_rate = infection_rate + recovery_rate
173
174         if total_rate <= 0.0
175             break
176         end
177
178         dt = rand(rng, Exponential(1.0 / total_rate))
179         next_t = t + dt
180
181         if next_t > params.tmax
182             break
183         end
184

```



```

185     t = next_t
186
187     if rand(rng) < infection_rate / total_rate && S > 0
188         person = random_person_with_status(rng, population, :S)
189
190         if person !== nothing
191             person.status = :I
192             add_event!(events, t, :infection, person.id, population)
193         end
194     else
195         person = random_person_with_status(rng, population, :I)
196
197         if person !== nothing
198             person.status = :R
199             add_event!(events, t, :recovery, person.id, population)
200         end
201     end
202
203     push_state!(time, S_values, I_values, R_values, t, population)
204 end
205
206 if time[end] < params.tmax
207     push_state!(time, S_values, I_values, R_values, params.tmax, population)
208 end
209
210 return SIRSimulationResult(params, time, S_values, I_values, R_values, events)
211 end
212

```

```

213 # Дискретно-событийная SIR-модель со сценарием вакцинации.
214 function simulate_sir_des_vaccination(
215     params::SIRParameters;
216     vaccination_time::Float64 = 10.0,
217     vaccination_fraction::Float64 = 0.2,
218 )
219     rng = MersenneTwister(params.seed)
220     population = initialize_population(params)
221
222     time = Float64[]
223     S_values = Int[]
224     I_values = Int[]
225     R_values = Int[]
226
227     events = DataFrame(
228         t = Float64[],
229         event = Symbol[],
230         person_id = Int[],
231         S = Int[],
232         I = Int[],
233         R = Int[],
234     )
235
236     t = 0.0
237     N = params.S0 + params.I0 + params.R0
238     vaccination_done = false
239
240     push_state!(time, S_values, I_values, R_values, t, population)

```

```

241
242 while t < params.tmax
243     S, I, _ = count_statuses(population)
244
245     if I == 0
246         break
247     end
248
249     infection_rate = params.beta * params.c * S * I / N
250     recovery_rate = params.gamma * I
251     total_rate = infection_rate + recovery_rate
252
253     if total_rate <= 0.0
254         break
255     end
256
257     dt = rand(rng, Exponential(1.0 / total_rate))
258     next_t = t + dt
259
260     # Вакцинация выполняется как отдельное событие.
261     if !vaccination_done &&
262         t < vaccination_time &&
263         vaccination_time <= next_t &&
264         vaccination_time <= params.tmax
265
266         t = vaccination_time
267
268         susceptible_indexes = findall(

```

```

269         person -> person.status == :S,
270         population,
271     )
272
273     vaccination_fraction = clamp(vaccination_fraction, 0.0, 1.0)
274     vaccinated_count = floor(
275         Int,
276         vaccination_fraction * length(susceptible_indexes),
277     )
278
279     if vaccinated_count > 0
280         selected_indexes = rand(
281             rng,
282             susceptible_indexes,
283             vaccinated_count,
284         )
285
286         for index in selected_indexes
287             population[index].status = :R
288         end
289     end
290
291     vaccination_done = true
292
293     add_event!(events, t, :vaccination, 0, population)
294     push_state!(time, S_values, I_values, R_values, t, population)
295
296     continue

```

```

297         end
298
299         if next_t > params.tmax
300             break
301         end
302
303         t = next_t
304
305         if rand(rng) < infection_rate / total_rate && S > 0
306             person = random_person_with_status(rng, population, :S)
307
308             if person !== nothing
309                 person.status = :I
310                 add_event!(events, t, :infection, person.id, population)
311             end
312         else
313             person = random_person_with_status(rng, population, :I)
314
315             if person !== nothing
316                 person.status = :R
317                 add_event!(events, t, :recovery, person.id, population)
318             end
319         end
320
321         push_state!(time, S_values, I_values, R_values, t, population)
322     end
323
324     if time[end] < params.tmax

```

```

325         push_state!(time, S_values, I_values, R_values, params.tmax, population)
326     end
327
328     return SIRSimulationResult(params, time, S_values, I_values, R_values, events)
329 end
330
331 # Таблица временных рядов S, I и R.
332 function result_dataframe(result::SIRSimulationResult)
333     return DataFrame(
334         t = result.time,
335         S = result.S,
336         I = result.I,
337         R = result.R,
338     )
339 end
340
341 # Таблица событий модели.
342 function events_dataframe(result::SIRSimulationResult)
343     return result.events
344 end
345
346 # Краткая итоговая таблица по одному запуску.
347 function summary_dataframe(result::SIRSimulationResult)
348     params = result.params
349
350     peak_I, peak_index = findmax(result.I)
351     peak_time = result.time[peak_index]
352

```

```

353     final_S = result.S[end]
354     final_I = result.I[end]
355     final_R = result.R[end]
356
357     return DataFrame(
358         S0 = [params.S0],
359         I0 = [params.I0],
360         R0 = [params.R0],
361         beta = [params.beta],
362         c = [params.c],
363         gamma = [params.gamma],
364         reproduction_number = [sir_basic_reproduction_number(params)],
365         tmax = [params.tmax],
366         seed = [params.seed],
367         peak_I = [peak_I],
368         peak_time = [peak_time],
369         final_S = [final_S],
370         final_I = [final_I],
371         final_R = [final_R],
372         final_size = [final_R - params.R0],
373         epidemic_duration = [result.time[end]],
374         events_count = [nrow(result.events)],
375     )
376 end
377
378 end

```

В source-файле реализована функция `simulate_sir_des`, которая выполняет стохастическое дискретно-событийное моделирование. На каждом шаге рассчитываются

интенсивности заражения и выздоровления, затем случайным образом определяется время до следующего события и тип события. После этого состояние популяции обновляется, а значения S , I , R сохраняются во временные ряды.

Также в source-модуль добавлена функция `simulate_sir_des_vaccination`, которая моделирует вакцинацию: в заданный момент времени часть восприимчивых агентов переводится из состояния S в состояние R .

5 Базовый запуск DES SIR-модели

5.1 Скрипт `sir_des.jl`

Файл `scripts/sir_des.jl` выполняет базовый запуск модели. В нём задаются начальные параметры, запускается симуляция, формируются таблицы и строятся основные графики.

```
1 using DrWatson
2 @quickactivate "project"
3
4 include(scrdir("sir_model.jl"))
5 using .SIRModel
6
7 using CSV
8 using DataFrames
9 using Plots
10
11 # Параметры базового запуска SIR-модели.
12 params = SIRParameters(
13     990,      # S0
14     10,       # I0
15     0,        # R0
16     0.05,     # beta
```

```

17     10.0,      # c
18     0.25,      # gamma
19     100.0,     # tmax
20     123,       # seed
21 )
22
23 # Запуск дискретно-событийной симуляции.
24 result = simulate_sir_des(params)
25
26 # Формирование таблиц результатов.
27 df_result = result_dataframe(result)
28 df_events = events_dataframe(result)
29 df_summary = summary_dataframe(result)
30
31 # Сохранение результатов в data/.
32 CSV.write(datadir("sir_des.csv"), df_result)
33 CSV.write(datadir("sir_des_events.csv"), df_events)
34 CSV.write(datadir("sir_des_summary.csv"), df_summary)
35
36 println("SIR DES summary:")
37 println(df_summary)
38
39 # График динамики S, I и R.
40 p_dynamics = plot(
41     df_result.t,
42     [df_result.S df_result.I df_result.R],
43     label = ["S" "I" "R"],
44     xlabel = "Time",

```

```

45     ylabel = "Population",
46     title = "SIR DES dynamics",
47     linewidth = 2,
48 )
49
50 savefig(p_dynamics, plotsdir("sir_des_dynamics.png"))
51
52 # График только для числа инфицированных.
53 p_infected = plot(
54     df_result.t,
55     df_result.I,
56     label = "Infected",
57     xlabel = "Time",
58     ylabel = "Infected",
59     title = "SIR DES infected dynamics",
60     linewidth = 2,
61 )
62
63 savefig(p_infected, plotsdir("sir_des_infected.png"))
64
65 # Столбчатая диаграмма итогового состояния.
66 final_state = DataFrame(
67     group = ["S", "I", "R"],
68     value = [
69         df_result.S[end],
70         df_result.I[end],
71         df_result.R[end],
72     ],

```

```

73 )
74
75 CSV.write(datadir("sir_des_final_state.csv"), final_state)
76
77 p_final = bar(
78     final_state.group,
79     final_state.value,
80     label = "Final state",
81     xlabel = "Group",
82     ylabel = "Population",
83     title = "SIR DES final state",
84 )
85
86 savefig(p_final, plotsdir("sir_des_final_state.png"))
87
88 println()
89 println("SIR DES simulation completed.")
90 println("Saved data:")
91 println("  data/sir_des.csv")
92 println("  data/sir_des_events.csv")
93 println("  data/sir_des_summary.csv")
94 println("  data/sir_des_final_state.csv")
95 println("Saved plots:")
96 println("  plots/sir_des_dynamics.png")
97 println("  plots/sir_des_infected.png")
98 println("  plots/sir_des_final_state.png")

```

Для базового запуска были использованы параметры:

Параметр	Значение	Смысл
S0	990	начальное число восприимчивых
I0	10	начальное число инфицированных
R0	0	начальное число выздоровевших
beta	0.05	вероятность передачи инфекции
c	10.0	среднее число контактов
gamma	0.25	интенсивность выздоровления
tmax	100.0	время моделирования
seed	123	начальное значение генератора случайных чисел

При таких параметрах базовое репродуктивное число равно:

$$R_0 = \frac{\beta c}{\gamma} = \frac{0.05 \cdot 10}{0.25} = 2.$$

Так как $R_0 > 1$, инфекция в начале моделирования распространяется, а не затухает сразу.

5.2 Выполнение базового скрипта

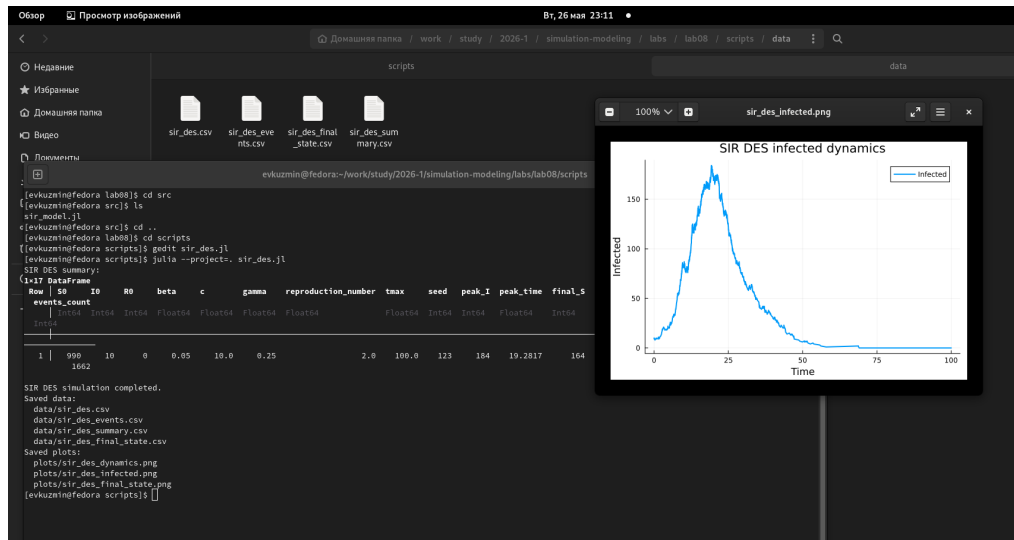


Рисунок 5.1: Выполнение и результаты базового скрипта

По результатам выполнения базового скрипта были сохранены таблицы с временными рядами, событиями и итоговой сводкой. Также были построены графики динамики S, I, R, отдельная динамика инфицированных и финальное состояние системы.

5.3 Динамика SIR-модели

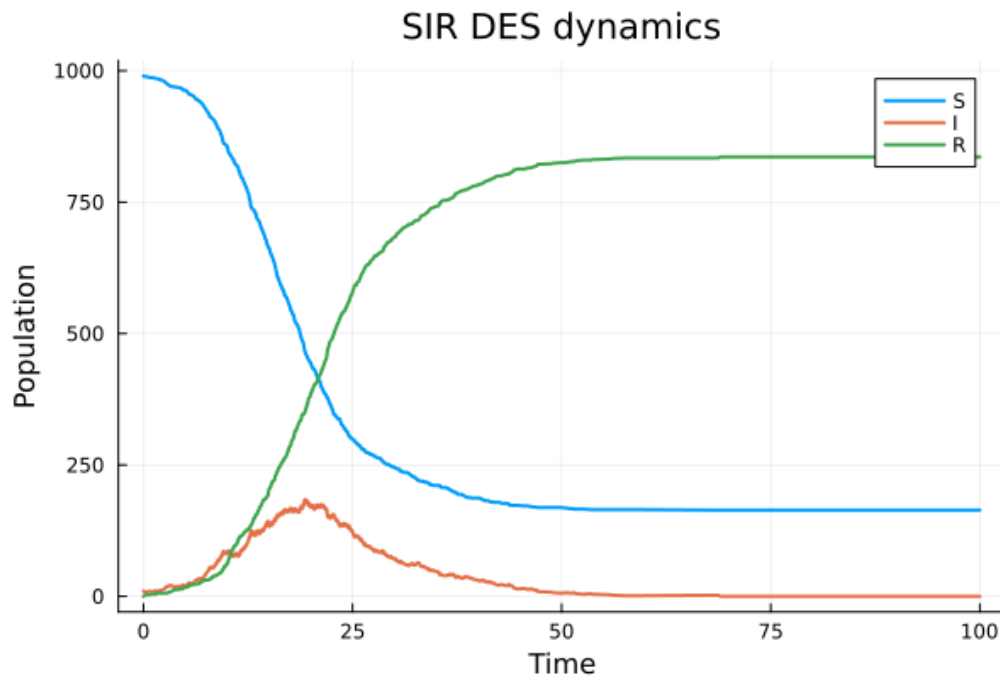


Рисунок 5.2: SIR DES dynamics

На графике общей динамики видно классическое поведение SIR-модели. Число восприимчивых агентов S постепенно уменьшается, так как часть агентов заражается и переходит в состояние I . Число инфицированных сначала растёт, затем достигает пика и после этого снижается. Число агентов в состоянии R монотонно увеличивается, поскольку выздоровевшие агенты больше не участвуют в распространении инфекции.

5.4 Динамика инфицированных

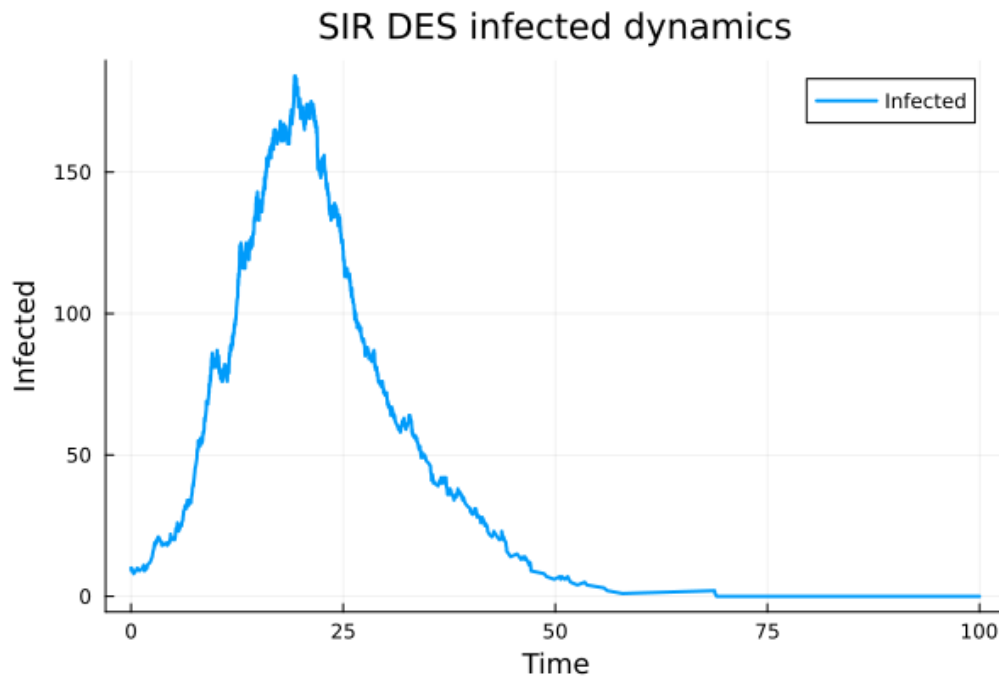


Рисунок 5.3: SIR DES infected dynamics

Отдельный график инфицированных показывает эпидемическую волну. На начальном участке число инфицированных растёт, затем достигает максимума, после чего начинает снижаться. Снижение связано с тем, что восприимчивых агентов становится меньше, а инфицированные постепенно переходят в состояние R.

5.5 Финальное состояние

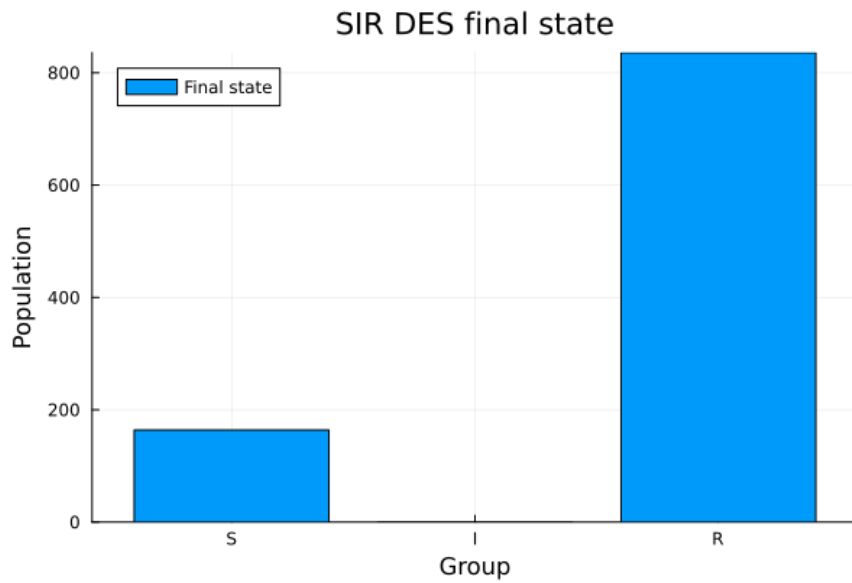


Рисунок 5.4: SIR DES final state

Финальное состояние показывает распределение агентов по группам в конце моделирования. В базовом сценарии активных инфицированных к концу моделирования практически не остаётся, а основная часть популяции переходит в состояние R. Это означает, что эпидемия завершилась естественным образом.

6 Literate-версия первого скрипта

Для базового запуска была подготовлена literate-версия `scripts/sir_des_literate.jl`. Она содержит тот же вычислительный сценарий, но оформлена как пояснительный документ: перед блоками кода добавлены текстовые комментарии, описывающие смысл выполняемых действий.

```
1  # # Лабораторная работа №8
2  #
3  # ## Базовая дискретно-событийная SIR-модель
4  #
5  # В данном скрипте выполняется базовый запуск SIR-модели в
6  # дискретно-событийном подходе.
7  #
8  # Модель SIR разделяет популяцию на три группы:
9  #
10 # - `S` — восприимчивые агенты;
11 # - `I` — инфицированные агенты;
12 # - `R` — выздоровевшие или удалённые из процесса распространения инфекции.
13 #
14 # В отличие от непрерывной модели на дифференциальных уравнениях,
15 # в дискретно-событийной модели состояние системы меняется только
16 # в моменты отдельных событий. В данной реализации такими событиями
17 # являются заражение и выздоровление.
```

```

18
19 using DrWatson
20 @quickactivate "project"
21
22 include(scrdir("sir_model.jl"))
23 using .SIRModel
24
25 using CSV
26 using DataFrames
27 using Plots
28 using Literate
29
30 # ## Задание параметров модели
31 #
32 # Зададим параметры базового эксперимента.
33 #
34 # В начальный момент времени в популяции находится 990 восприимчивых
35 # агентов, 10 инфицированных и 0 выздоровевших. Параметр `beta`
36 # задаёт вероятность передачи инфекции при контакте, `c` задаёт
37 # среднее число контактов, а `gamma` – интенсивность выздоровления.
38
39 params = SIRParameters(
40     990,      # начальное число восприимчивых
41     10,       # начальное число инфицированных
42     0,        # начальное число выздоровевших
43     0.05,     # вероятность передачи инфекции
44     10.0,     # среднее число контактов
45     0.25,     # интенсивность выздоровления

```

```

46     100.0,    # максимальное время моделирования
47     123,      # seed
48 )
49
50 # Для выбранных параметров можно вычислить базовое репродуктивное число.
51 # Оно показывает, сколько человек в среднем может заразить один
52 # инфицированный агент в полностью восприимчивой популяции.
53
54 R0_value = sir_basic_reproduction_number(params)
55
56 println("Basic reproduction number R0 = ", R0_value)
57
58 # ## Запуск дискретно-событийной симуляции
59 #
60 # После задания параметров запускаем модель. Внутри функции
61 # `simulate_sir_des` выполняется дискретно-событийная симуляция:
62 #
63 # - рассчитываются интенсивности заражения и выздоровления;
64 # - случайно определяется время до следующего события;
65 # - выбирается тип события;
66 # - состояние популяции обновляется;
67 # - результат сохраняется во временные ряды и таблицу событий.
68
69 result = simulate_sir_des(params)
70
71 # ## Формирование таблиц
72 #
73 # После выполнения симуляции сформируем три основные таблицы:

```

```

74 #
75 # - временной ряд значений `S`, `I`, `R`;
76 # - таблицу событий;
77 # - итоговую сводную таблицу по запуску.
78
79 df_result = result_dataframe(result)
80 df_events = events_dataframe(result)
81 df_summary = summary_dataframe(result)
82
83 # Сохраним полученные данные в каталог `data/`.
84
85 CSV.write(datadir("sir_des_literate.csv"), df_result)
86 CSV.write(datadir("sir_des_literate_events.csv"), df_events)
87 CSV.write(datadir("sir_des_literate_summary.csv"), df_summary)
88
89 println("SIR DES literate summary:")
90 println(df_summary)
91
92 # ## Визуализация динамики SIR
93 #
94 # Первый график показывает общую динамику трёх групп: `S`, `I` и `R`.
95 # По нему можно увидеть, как число восприимчивых уменьшается, число
96 # инфицированных сначала растёт, а затем снижается, а число выздоровевших
97 # постепенно увеличивается.
98
99 p_dynamics = plot(
100     df_result.t,
101     [df_result.S df_result.I df_result.R],

```

```

102     label = ["S" "I" "R"],
103     xlabel = "Time",
104     ylabel = "Population",
105     title = "SIR DES dynamics",
106     linewidth = 2,
107 )
108
109 savefig(p_dynamics, plotsdir("sir_des_literate_dynamics.png"))
110
111 # ## Динамика инфицированных
112 #
113 # Второй график отдельно показывает число инфицированных агентов.
114 # Он удобен для анализа пика эпидемии и времени, в которое этот пик
115 # достигается.
116
117 p_infected = plot(
118     df_result.t,
119     df_result.I,
120     label = "Infected",
121     xlabel = "Time",
122     ylabel = "Infected",
123     title = "SIR DES infected dynamics",
124     linewidth = 2,
125 )
126
127 savefig(p_infected, plotsdir("sir_des_literate_infected.png"))
128
129 # ## Финальное состояние системы

```

```

130 #
131 # Дополнительно построим столбчатую диаграмму финального состояния.
132 # Она показывает, сколько агентов осталось в группах `S`, `I` и `R`
133 # к концу моделирования.
134
135 final_state = DataFrame(
136     group = ["S", "I", "R"],
137     value = [
138         df_result.S[end],
139         df_result.I[end],
140         df_result.R[end],
141     ],
142 )
143
144 CSV.write(datadir("sir_des_literate_final_state.csv"), final_state)
145
146 p_final = bar(
147     final_state.group,
148     final_state.value,
149     label = "Final state",
150     xlabel = "Group",
151     ylabel = "Population",
152     title = "SIR DES final state",
153 )
154
155 savefig(p_final, plotsdir("sir_des_literate_final_state.png"))
156
157 # ## Итог базового запуска

```

```

158 #
159 # В результате выполнения literate-версии были получены таблицы и графики,
160 # аналогичные обычному базовому запуску. Отличие этой версии заключается
161 # в том, что код оформлен в литературном стиле и может быть преобразован
162 # в чистый Julia-скрипт, Jupyter Notebook и Quarto-документ.
163
164 println()
165 println("SIR DES literate simulation completed.")
166 println("Saved data:")
167 println("  data/sir_des_literate.csv")
168 println("  data/sir_des_literate_events.csv")
169 println("  data/sir_des_literate_summary.csv")
170 println("  data/sir_des_literate_final_state.csv")
171 println("Saved plots:")
172 println("  plots/sir_des_literate_dynamics.png")
173 println("  plots/sir_des_literate_infected.png")
174 println("  plots/sir_des_literate_final_state.png")
175
176 # ## Генерация файлов из literate-версии
177 #
178 # В конце скрипта выполняется генерация производных файлов:
179 #
180 # - чистого Julia-скрипта;
181 # - Jupyter Notebook;
182 # - Quarto-документа.
183
184 output_dir = scriptsdir("generated", "sir_des_literate")
185 mkpath(output_dir)

```



```

186
187 Literate.script(@__FILE__, output_dir)
188 Literate.notebook(@__FILE__, output_dir)
189 Literate.markdown(@__FILE__, output_dir; flavor = Literate.QuartoFlavor())
190
191 println()
192 println("Generated literate outputs:")
193 println("  scripts/generated/sir_des_literate/")

```

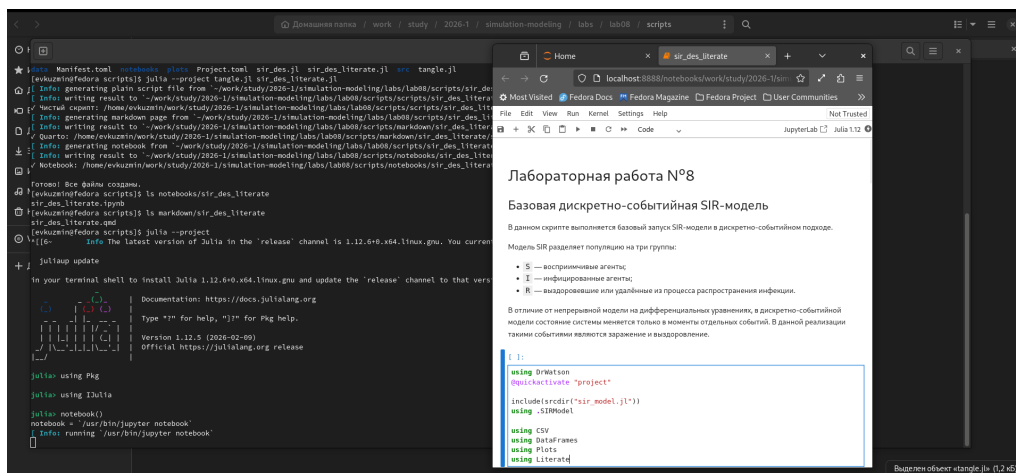


Рисунок 6.1: Создание и notebook literate-версии первого скрипта

Literate-версия нужна для того, чтобы из одного файла можно было получить чистый Julia-скрипт, Jupyter Notebook и Quarto-документ. Генерация производных файлов выполнялась отдельно через `tangle.jl`.

7 Параметризованная версия первого скрипта

7.1 Скрипт `sir_des_params.jl`

Параметризованная версия первого скрипта предназначена для анализа чувствительности модели к основным параметрам SIR-модели: `beta`, `c` и `gamma`.

```
1 # # Лабораторная работа №8
2 #
3 # ## Параметризованная версия DES SIR-модели
4 #
5 # В данном скрипте выполняется параметризованный анализ базовой
6 # дискретно-событийной SIR-модели.
7 #
8 # В отличие от обычного запуска `sir_des.jl`, здесь модель запускается
9 # несколько раз при разных значениях параметров. Это позволяет оценить,
10 # как параметры заражения, контактов и выздоровления влияют на динамику
11 # эпидемии.
12 #
13 # В работе исследуются три параметра:
14 #
15 # - `beta` — вероятность передачи инфекции при контакте;
```

```

16 # - `c` — среднее число контактов;
17 # - `gamma` — интенсивность выздоровления.
18 #
19 # Для каждого сценария выполняется несколько повторов с разными `seed`,
20 # так как дискретно-событийная модель является стохастической.
21
22 using DrWatson
23 @quickactivate "project"
24
25 include(scrdir("sir_model.jl"))
26 using .SIRModel
27
28 using CSV
29 using DataFrames
30 using Plots
31 using Statistics
32 using Literate
33
34 # ## Базовые параметры
35 #
36 # Сначала зададим базовые параметры модели. Они соответствуют начальному
37 # сценарию, относительно которого далее будут изменяться отдельные параметры.
38
39 base_S0 = 990
40 base_I0 = 10
41 base_R0 = 0
42 base_tmax = 100.0
43

```

```

44 base_beta = 0.05
45 base_c = 10.0
46 base_gamma = 0.25
47
48 replications = 5
49
50 # ## Формирование сценариев
51 #
52 # Для параметризованного анализа сформируем таблицу сценариев.
53 #
54 # В первой группе меняется `beta`, во второй группе меняется `c`,
55 # в третьей группе меняется `gamma`. Остальные параметры при этом
56 # остаются базовыми.
57
58 scenarios = DataFrame(
59     scenario = String[],
60     beta = Float64[],
61     c = Float64[],
62     gamma = Float64[],
63 )
64
65 # Изменение вероятности передачи инфекции.
66 for beta in [0.03, 0.05, 0.07]
67     push!(
68         scenarios,
69         (
70             scenario = "beta_scan",
71             beta = beta,

```

```

72         c = base_c,
73         gamma = base_gamma,
74     ),
75 )
76 end
77
78 # Изменение среднего числа контактов.
79 for c in [6.0, 10.0, 14.0]
80     push!(
81         scenarios,
82         (
83             scenario = "contact_scan",
84             beta = base_beta,
85             c = c,
86             gamma = base_gamma,
87         ),
88     )
89 end
90
91 # Изменение интенсивности выздоровления.
92 for gamma in [0.15, 0.25, 0.35]
93     push!(
94         scenarios,
95         (
96             scenario = "gamma_scan",
97             beta = base_beta,
98             c = base_c,
99             gamma = gamma,

```

```

100     ),
101     )
102 end
103
104 # ## Запуск серии экспериментов
105 #
106 # Для каждого сценария выполняется несколько независимых повторов.
107 # Это нужно для того, чтобы сгладить случайность отдельных запусков.
108 #
109 # В таблицу `all_results` сохраняются временные ряды, а в таблицу
110 # `all_summaries` – итоговые характеристики каждого запуска.
111
112 all_results = DataFrame()
113 all_summaries = DataFrame()
114
115 for (scenario_id, row) in enumerate(eachrow(scenarios))
116     for replication in 1:replications
117         seed = 1000 + 100 * scenario_id + replication
118
119         params = SIRParameters(
120             base_S0,
121             base_I0,
122             base_R0,
123             row.beta,
124             row.c,
125             row.gamma,
126             base_tmax,
127             seed,

```

```

128         )
129
130         result = simulate_sir_des(params)
131
132         df_result = result_dataframe(result)
133         df_summary = summary_dataframe(result)
134
135         df_result.scenario .= row.scenario
136         df_result.beta .= row.beta
137         df_result.c .= row.c
138         df_result.gamma .= row.gamma
139         df_result.replication .= replication
140         df_result.seed .= seed
141
142         df_summary.scenario .= row.scenario
143         df_summary.replication .= replication
144
145         append!(all_results, df_result)
146         append!(all_summaries, df_summary)
147     end
148 end
149
150 # ## Сводная таблица
151 #
152 # После выполнения всех повторов сгруппируем результаты по сценариям.
153 # Для каждого набора параметров считаются средние значения и стандартные
154 # отклонения основных характеристик.
155

```

```

156 summary_by_scenario = combine(
157     groupby(all_summaries, [:scenario, :beta, :c, :gamma]),
158     :reproduction_number => mean => :mean_reproduction_number,
159     :peak_I => mean => :mean_peak_I,
160     :peak_I => std => :std_peak_I,
161     :peak_time => mean => :mean_peak_time,
162     :final_size => mean => :mean_final_size,
163     :final_size => std => :std_final_size,
164     :events_count => mean => :mean_events_count,
165 )
166
167 CSV.write(datadir("sir_des_params_results.csv"), all_results)
168 CSV.write(datadir("sir_des_params_summaries.csv"), all_summaries)
169 CSV.write(datadir("sir_des_params_summary_by_scenario.csv"), summary_by_scenario)
170
171 println("SIR DES parameter scan summary:")
172 println(summary_by_scenario)
173
174 # ## Отдельные таблицы по группам параметров
175 #
176 # Для удобства построения графиков разделим общую сводную таблицу
177 # на три группы: анализ `beta`, анализ `c` и анализ `gamma`.
178
179 beta_summary = filter(row -> row.scenario == "beta_scan", summary_by_scenario)
180 contact_summary = filter(row -> row.scenario == "contact_scan", summary_by_scenario)
181 gamma_summary = filter(row -> row.scenario == "gamma_scan", summary_by_scenario)
182
183 CSV.write(datadir("sir_des_params_beta_summary.csv"), beta_summary)

```



```

184 CSV.write(datadir("sir_des_params_contact_summary.csv"), contact_summary)
185 CSV.write(datadir("sir_des_params_gamma_summary.csv"), gamma_summary)
186
187 # ## Влияние beta на пик инфекции
188 #
189 # Параметр `beta` отвечает за вероятность передачи инфекции при контакте.
190 # При увеличении `beta` ожидается рост пика инфицированных и итогового
191 # размера эпидемии.
192
193 p_beta = plot(
194     beta_summary.beta,
195     beta_summary.mean_peak_I,
196     marker = :circle,
197     label = "Mean peak I",
198     xlabel = "Beta",
199     ylabel = "Peak infected",
200     title = "SIR DES: effect of beta on infection peak",
201     linewidth = 2,
202 )
203
204 savefig(p_beta, plotsdir("sir_des_params_beta_peak.png"))
205
206 # ## Влияние числа контактов
207 #
208 # Параметр `c` задаёт среднее число контактов. Чем больше контактов,
209 # тем чаще возникают возможности передачи инфекции.
210
211 p_contact = plot(

```

```

212     contact_summary.c,
213     contact_summary.mean_peak_I,
214     marker = :circle,
215     label = "Mean peak I",
216     xlabel = "Average contacts",
217     ylabel = "Peak infected",
218     title = "SIR DES: effect of contacts on infection peak",
219     linewidth = 2,
220 )
221
222 savefig(p_contact, plotsdir("sir_des_params_contacts_peak.png"))
223
224 # ## Влияние gamma
225 #
226 # Параметр `gamma` отвечает за интенсивность выздоровления.
227 # При увеличении `gamma` инфицированные быстрее переходят в состояние `R`,
228 # поэтому пик инфекции обычно становится ниже.
229
230 p_gamma = plot(
231     gamma_summary.gamma,
232     gamma_summary.mean_peak_I,
233     marker = :circle,
234     label = "Mean peak I",
235     xlabel = "Gamma",
236     ylabel = "Peak infected",
237     title = "SIR DES: effect of gamma on infection peak",
238     linewidth = 2,
239 )

```

```

240
241 savefig(p_gamma, plotsdir("sir_des_params_gamma_peak.png"))
242
243 # ## Итоговый размер эпидемии по сценариям
244 #
245 # Итоговый размер эпидемии показывает, сколько агентов перешло в состояние
246 # `R` за время моделирования. Этот показатель удобно сравнить для всех
247 # сценариев на одной диаграмме.
248
249 summary_labels = string.(
250     summary_by_scenario.scenario,
251     "\n $\beta$ =",
252     summary_by_scenario.beta,
253     ", c=",
254     summary_by_scenario.c,
255     ",  $\gamma$ =",
256     summary_by_scenario.gamma,
257 )
258
259 x_positions = collect(1:nrow(summary_by_scenario))
260
261 p_final_size = bar(
262     x_positions,
263     summary_by_scenario.mean_final_size,
264     label = "Mean final size",
265     xlabel = "Scenario",
266     ylabel = "Final size",
267     title = "SIR DES: final epidemic size by scenario",

```

```

268     xticks = (x_positions, summary_labels),
269     xrotation = 35,
270 )
271
272 savefig(p_final_size, plotsdir("sir_des_params_final_size.png"))
273
274 # ## Завершение параметризованного анализа
275 #
276 # В результате выполнения скрипта были получены таблицы всех запусков,
277 # сводные таблицы по сценариям и четыре графика влияния параметров
278 # на развитие эпидемии.
279
280 println()
281 println("SIR DES parameter scan completed.")
282 println("Saved data:")
283 println("  data/sir_des_params_results.csv")
284 println("  data/sir_des_params_summaries.csv")
285 println("  data/sir_des_params_summary_by_scenario.csv")
286 println("  data/sir_des_params_beta_summary.csv")
287 println("  data/sir_des_params_contact_summary.csv")
288 println("  data/sir_des_params_gamma_summary.csv")
289 println("Saved plots:")
290 println("  plots/sir_des_params_beta_peak.png")
291 println("  plots/sir_des_params_contacts_peak.png")
292 println("  plots/sir_des_params_gamma_peak.png")
293 println("  plots/sir_des_params_final_size.png")
294
295

```

В этом скрипте формируются несколько сценариев:

Группа сценариев	Изменяемый параметр	Значения
beta_scan	beta	0.03, 0.05, 0.07
contact_scan	c	6.0, 10.0, 14.0
gamma_scan	gamma	0.15, 0.25, 0.35

Для каждого сценария выполняется несколько повторов с разными seed, так как DES-модель является стохастической. После этого рассчитываются средние значения пика инфицированных, времени пика, итогового размера эпидемии и числа событий.

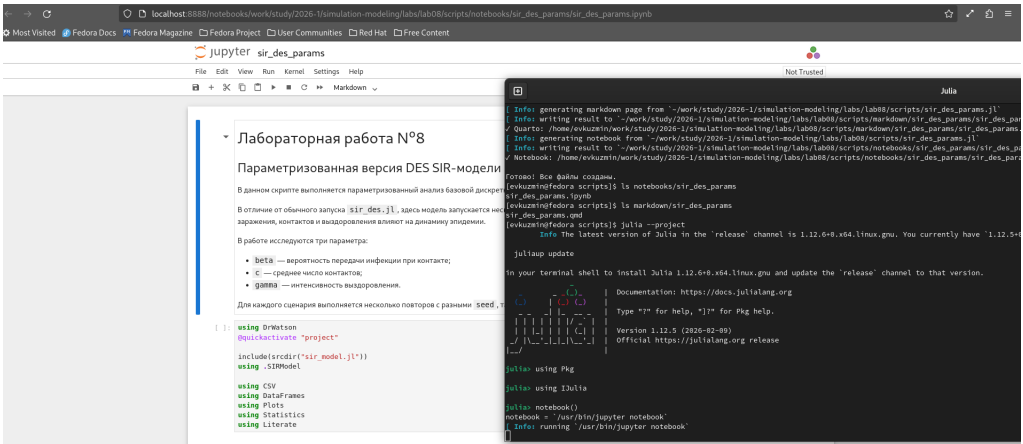


Рисунок 7.1: Создание параметризованной версии и notebook первого скрипта

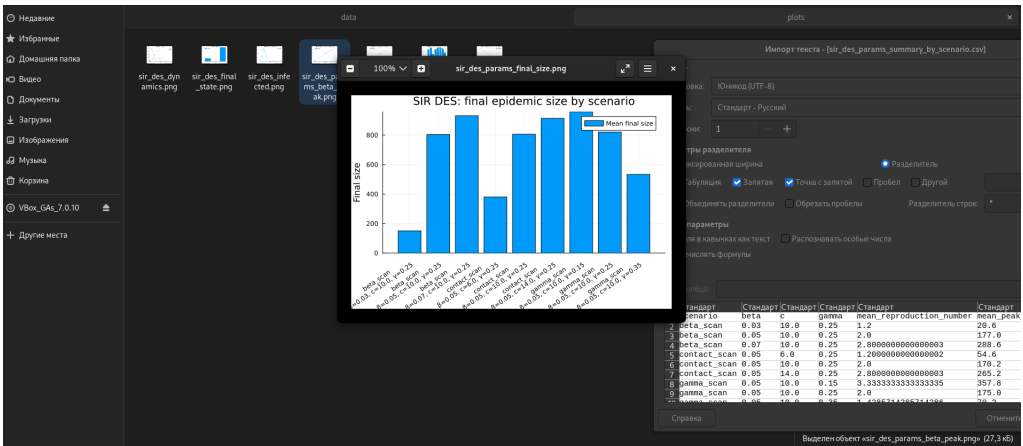


Рисунок 7.2: Результаты параметризованной версии первого скрипта

7.2 Влияние beta

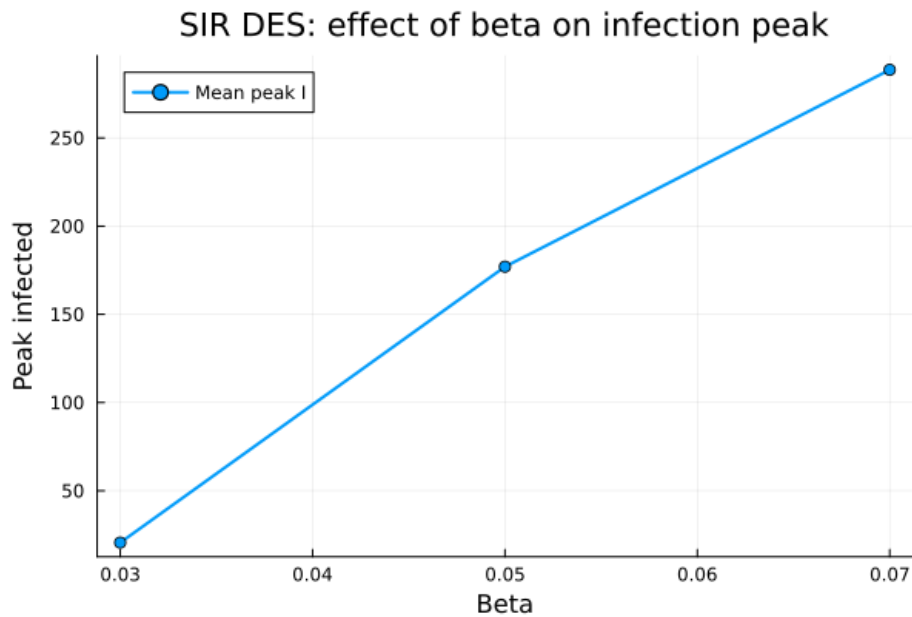


Рисунок 7.3: SIR DES: effect of beta on infection peak

Параметр β отвечает за вероятность передачи инфекции при контакте. На графике видно, что при увеличении β пик инфицированных возрастает. Это объясняется тем, что при большей вероятности передачи инфекции каждый контакт чаще приводит к заражению, поэтому эпидемия развивается быстрее и достигает более высокого максимума.

7.3 Влияние среднего числа контактов

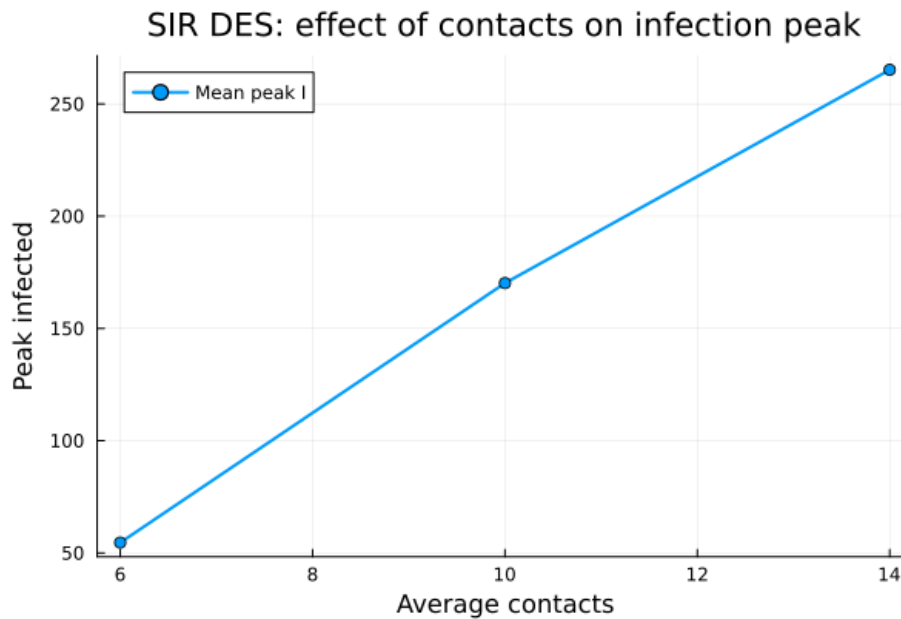


Рисунок 7.4: SIR DES: effect of contacts on infection peak

Параметр β задаёт среднее число контактов. При росте числа контактов максимальное количество инфицированных увеличивается. Это связано с тем, что большее число контактов повышает частоту потенциальных заражений.

7.4 Влияние гамма

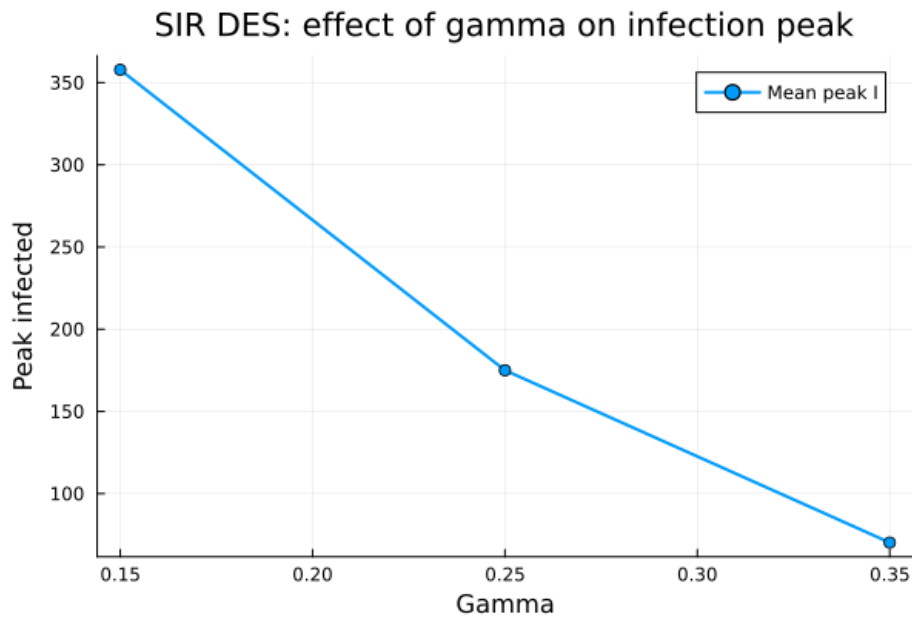


Рисунок 7.5: SIR DES: effect of gamma on infection peak

Параметр γ отвечает за интенсивность выздоровления. При увеличении γ максимальное число инфицированных снижается. Инфицированные агенты быстрее переходят в состояние R, меньше времени остаются источниками заражения и поэтому заражают меньше восприимчивых агентов.

7.5 Итоговый размер эпидемии по сценариям

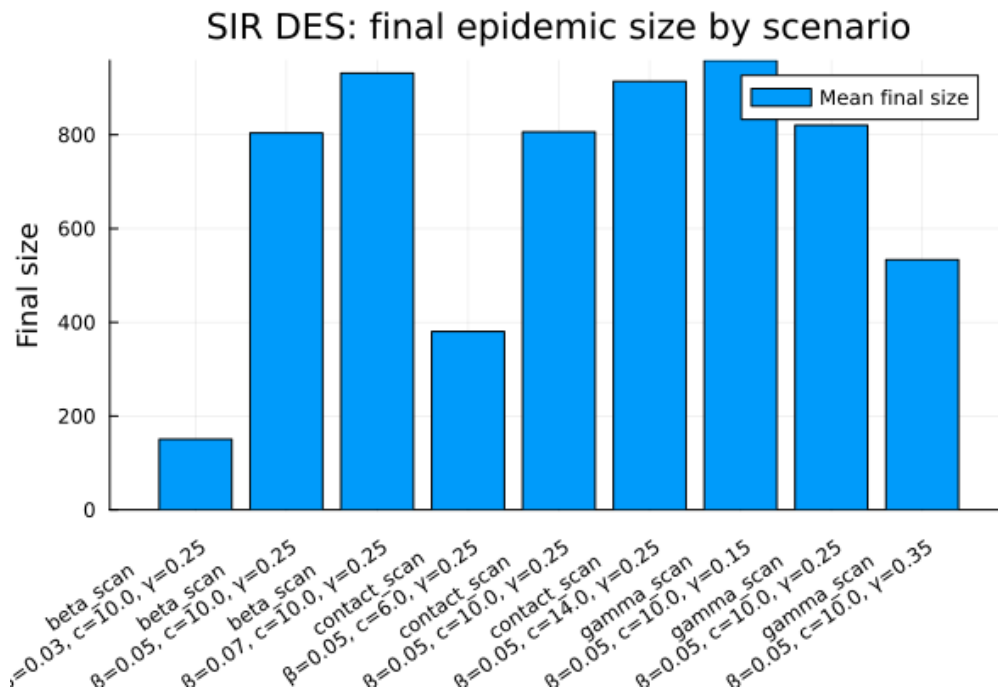


Рисунок 7.6: SIR DES: final epidemic size by scenario

Столбчатая диаграмма показывает итоговый размер эпидемии для разных сценариев. Наибольшие значения наблюдаются при увеличении вероятности передачи инфекции и числа контактов. При увеличении интенсивности выздоровления итоговый размер эпидемии уменьшается, так как инфицированные быстрее выходят из процесса распространения инфекции.

8 Аналитический скрипт

8.1 Скрипт `sir_des_analysis.jl`

Второй основной скрипт `scripts/sir_des_analysis.jl` выполняет расширенный анализ модели. В нём реализованы сравнение DES-модели с детерминированной SIR-моделью на ОДУ, сценарий вакцинации, вариант с фиксированной длительностью болезни и оценка производительности.

```
1 using DrWatson
2 @quickactivate "project"
3
4 include(scrdir("sir_model.jl"))
5 using .SIRModel
6
7 using CSV
8 using DataFrames
9 using Plots
10 using DifferentialEquations
11 using Random
12 using Distributions
13
14 # Базовые параметры модели.
15 params = SIRParameters(
```

```

16     990,      # S0
17     10,      # I0
18     0,       # R0
19     0.05,    # beta
20     10.0,    # c
21     0.25,    # gamma
22     100.0,   # tmax
23     123,     # seed
24 )
25
26 # Базовый запуск DES SIR.
27 des_result = simulate_sir_des(params)
28 df_des = result_dataframe(des_result)
29 df_des_summary = summary_dataframe(des_result)
30
31 # Детерминированная SIR-модель через систему ОДУ.
32 function sir_ode!(du, u, p, t)
33     beta, c, gamma, N = p
34
35     S = u[1]
36     I = u[2]
37     R = u[3]
38
39     infection = beta * c * S * I / N
40     recovery = gamma * I
41
42     du[1] = -infection
43     du[2] = infection - recovery

```

```

44     du[3] = recovery
45
46     return nothing
47 end
48
49 N = params.S0 + params.I0 + params.R0
50 u0 = [params.S0, params.I0, params.R0]
51 p = [params.beta, params.c, params.gamma, N]
52 tspan = (0.0, params.tmax)
53
54 ode_problem = ODEProblem(sir_ode!, u0, tspan, p)
55 ode_solution = solve(ode_problem, Tsit5(); saveat = 0.5)
56
57 ode_values = Array(ode_solution)
58
59 df_ode = DataFrame(
60     t = Float64.(ode_solution.t),
61     S = ode_values[1, :],
62     I = ode_values[2, :],
63     R = ode_values[3, :],
64 )
65
66 ode_peak_I, ode_peak_index = findmax(df_ode.I)
67 ode_peak_time = df_ode.t[ode_peak_index]
68
69 df_ode_summary = DataFrame(
70     model = ["ODE"],
71     peak_I = [ode_peak_I],

```

```

72     peak_time = [ode_peak_time],
73     final_S = [df_ode.S[end]],
74     final_I = [df_ode.I[end]],
75     final_R = [df_ode.R[end]],
76     final_size = [df_ode.R[end] - params.R0],
77 )
78
79 # DES SIR со сценарием вакцинации.
80 vaccination_result = simulate_sir_des_vaccination(
81     params;
82     vaccination_time = 10.0,
83     vaccination_fraction = 0.25,
84 )
85
86 df_vaccination = result_dataframe(vaccination_result)
87 df_vaccination_summary = summary_dataframe(vaccination_result)
88
89 # DES SIR с детерминированной длительностью болезни.
90 function simulate_sir_des_fixed_recovery(
91     params::SIRParameters;
92     illness_duration::Float64 = 1.0 / params.gamma,
93 )
94     rng = MersenneTwister(params.seed)
95
96     S = params.S0
97     I = params.I0
98     R = params.R0
99

```

```

100     N = S + I + R
101     t = 0.0
102
103     time = Float64[t]
104     S_values = Int[S]
105     I_values = Int[I]
106     R_values = Int[R]
107
108     events = DataFrame(
109         t = Float64[],
110         event = Symbol[],
111         S = Int[],
112         I = Int[],
113         R = Int[],
114     )
115
116     # Для начальных инфицированных заранее задаём фиксированное время выздоровления.
117     recovery_times = fill(illness_duration, I)
118
119     while t < params.tmax
120         if I == 0 && isempty(recovery_times)
121             break
122         end
123
124         infection_rate = params.beta * params.c * S * I / N
125
126         if infection_rate > 0.0 && S > 0
127             next_infection_t = t + rand(rng, Exponential(1.0 / infection_rate))

```

```

128     else
129         next_infection_t = Inf
130     end
131
132     if isempty(recovery_times)
133         next_recovery_t, recovery_index = findmin(recovery_times)
134     else
135         next_recovery_t = Inf
136         recovery_index = 0
137     end
138
139     next_t = min(next_infection_t, next_recovery_t)
140
141     if next_t == Inf || next_t > params.tmax
142         break
143     end
144
145     t = next_t
146
147     if next_recovery_t <= next_infection_t
148         I -= 1
149         R += 1
150         deleteat!(recovery_times, recovery_index)
151
152         push!(
153             events,
154             (
155                 t = t,

```

```

156         event = :fixed_recovery,
157         S = S,
158         I = I,
159         R = R,
160     ),
161 )
162 else
163     S -= 1
164     I += 1
165     push!(recovery_times, t + illness_duration)
166
167     push!(
168         events,
169         (
170             t = t,
171             event = :infection,
172             S = S,
173             I = I,
174             R = R,
175         ),
176     )
177 end
178
179 push!(time, t)
180 push!(S_values, S)
181 push!(I_values, I)
182 push!(R_values, R)
183 end

```



```

184
185     if time[end] < params.tmax
186         push!(time, params.tmax)
187         push!(S_values, S)
188         push!(I_values, I)
189         push!(R_values, R)
190     end
191
192     df_result = DataFrame(
193         t = time,
194         S = S_values,
195         I = I_values,
196         R = R_values,
197     )
198
199     peak_I, peak_index = findmax(df_result.I)
200     peak_time = df_result.t[peak_index]
201
202     df_summary = DataFrame(
203         model = ["DES fixed recovery"],
204         peak_I = [Float64(peak_I)],
205         peak_time = [Float64(peak_time)],
206         final_S = [Float64(df_result.S[end])],
207         final_I = [Float64(df_result.I[end])],
208         final_R = [Float64(df_result.R[end])],
209         final_size = [Float64(df_result.R[end] - params.R0)],
210         events_count = [nrow(events)],
211         illness_duration = [illness_duration],

```

```

212     )
213
214     return df_result, events, df_summary
215 end
216
217 df_fixed, df_fixed_events, df_fixed_summary = simulate_sir_des_fixed_recovery(
218     params;
219     illness_duration = 1.0 / params.gamma,
220 )
221
222 # Формирование общей таблицы сравнения.
223 df_des_summary.model .= "DES base"
224 df_vaccination_summary.model .= "DES vaccination"
225
226 df_comparison = DataFrame(
227     model = String[],
228     peak_I = Float64[],
229     peak_time = Float64[],
230     final_S = Float64[],
231     final_I = Float64[],
232     final_R = Float64[],
233     final_size = Float64[],
234 )
235
236 push!(
237     df_comparison,
238     (
239         model = "DES base",

```

```

240     peak_I = Float64(df_des_summary.peak_I[1]),
241     peak_time = Float64(df_des_summary.peak_time[1]),
242     final_S = Float64(df_des_summary.final_S[1]),
243     final_I = Float64(df_des_summary.final_I[1]),
244     final_R = Float64(df_des_summary.final_R[1]),
245     final_size = Float64(df_des_summary.final_size[1]),
246 ),
247 )
248
249 push!(
250     df_comparison,
251     (
252         model = "ODE",
253         peak_I = Float64(df_ode_summary.peak_I[1]),
254         peak_time = Float64(df_ode_summary.peak_time[1]),
255         final_S = Float64(df_ode_summary.final_S[1]),
256         final_I = Float64(df_ode_summary.final_I[1]),
257         final_R = Float64(df_ode_summary.final_R[1]),
258         final_size = Float64(df_ode_summary.final_size[1]),
259     ),
260 )
261
262 push!(
263     df_comparison,
264     (
265         model = "DES vaccination",
266         peak_I = Float64(df_vaccination_summary.peak_I[1]),
267         peak_time = Float64(df_vaccination_summary.peak_time[1]),

```

```

268         final_S = Float64(df_vaccination_summary.final_S[1]),
269         final_I = Float64(df_vaccination_summary.final_I[1]),
270         final_R = Float64(df_vaccination_summary.final_R[1]),
271         final_size = Float64(df_vaccination_summary.final_size[1]),
272     ),
273 )
274
275 push!(
276     df_comparison,
277     (
278         model = "DES fixed recovery",
279         peak_I = Float64(df_fixed_summary.peak_I[1]),
280         peak_time = Float64(df_fixed_summary.peak_time[1]),
281         final_S = Float64(df_fixed_summary.final_S[1]),
282         final_I = Float64(df_fixed_summary.final_I[1]),
283         final_R = Float64(df_fixed_summary.final_R[1]),
284         final_size = Float64(df_fixed_summary.final_size[1]),
285     ),
286 )
287
288 # Оценка производительности при разных размерах популяции.
289 population_sizes = [500, 1000, 2000, 5000]
290
291 df_performance = DataFrame(
292     population_size = Int[],
293     elapsed_time = Float64[],
294     peak_I = Int[],
295     final_size = Int[],

```

```

296     events_count = Int[],
297 )
298
299 for population_size in population_sizes
300     test_params = SIRParameters(
301         population_size - 10,
302         10,
303         0,
304         params.beta,
305         params.c,
306         params.gamma,
307         params.tmax,
308         5000 + population_size,
309     )
310
311     test_result = nothing
312
313     elapsed_time = @elapsed begin
314         test_result = simulate_sir_des(test_params)
315     end
316
317     test_summary = summary_dataframe(test_result)
318
319     push!(
320         df_performance,
321         (
322             population_size = population_size,
323             elapsed_time = elapsed_time,

```

```

324         peak_I = test_summary.peak_I[1],
325         final_size = test_summary.final_size[1],
326         events_count = test_summary.events_count[1],
327     ),
328 )
329 end
330
331 # Сохранение таблиц.
332 CSV.write(datadir("sir_des_analysis_des.csv"), df_des)
333 CSV.write(datadir("sir_des_analysis_ode.csv"), df_ode)
334 CSV.write(datadir("sir_des_analysis_vaccination.csv"), df_vaccination)
335 CSV.write(datadir("sir_des_analysis_fixed_recovery.csv"), df_fixed)
336 CSV.write(datadir("sir_des_analysis_fixed_recovery_events.csv"), df_fixed_events)
337 CSV.write(datadir("sir_des_analysis_comparison.csv"), df_comparison)
338 CSV.write(datadir("sir_des_analysis_performance.csv"), df_performance)
339
340 println("SIR DES analysis comparison:")
341 println(df_comparison)
342
343 println()
344 println("SIR DES performance:")
345 println(df_performance)
346
347 # График 1. Сравнение DES и ODE по инфицированным.
348 p_compare_infected = plot(
349     df_des.t,
350     df_des.I,
351     label = "DES infected",

```

```

352     xlabel = "Time",
353     ylabel = "Infected",
354     title = "SIR: DES and ODE infected comparison",
355     linewidth = 2,
356 )
357
358 plot!(
359     p_compare_infected,
360     df_ode.t,
361     df_ode.I,
362     label = "ODE infected",
363     linewidth = 2,
364 )
365
366 savefig(p_compare_infected, plotsdir("sir_des_analysis_des_vs_ode.png"))
367
368 # График 2. Сравнение базового сценария и вакцинации.
369 p_vaccination = plot(
370     df_des.t,
371     df_des.I,
372     label = "Base infected",
373     xlabel = "Time",
374     ylabel = "Infected",
375     title = "SIR DES: base and vaccination scenarios",
376     linewidth = 2,
377 )
378
379 plot!(

```

```

380     p_vaccination,
381     df_vaccination.t,
382     df_vaccination.I,
383     label = "Vaccination infected",
384     linewidth = 2,
385 )
386
387 savefig(p_vaccination, plotsdir("sir_des_analysis_vaccination.png"))
388
389 # График 3. Сравнение стохастического и фиксированного выздоровления.
390 p_fixed = plot(
391     df_des.t,
392     df_des.I,
393     label = "Stochastic recovery",
394     xlabel = "Time",
395     ylabel = "Infected",
396     title = "SIR DES: stochastic and fixed recovery",
397     linewidth = 2,
398 )
399
400 plot!(
401     p_fixed,
402     df_fixed.t,
403     df_fixed.I,
404     label = "Fixed recovery",
405     linewidth = 2,
406 )
407

```



```

408 savefig(p_fixed, plotsdir("sir_des_analysis_fixed_recovery.png"))
409
410 # График 4. Сравнение пика инфекции.
411 x_positions = collect(1:nrow(df_comparison))
412
413 p_peak_comparison = bar(
414     x_positions,
415     df_comparison.peak_I,
416     label = "Peak infected",
417     xlabel = "Model",
418     ylabel = "Peak infected",
419     title = "SIR: infection peak comparison",
420     xticks = (x_positions, df_comparison.model),
421     xrotation = 25,
422 )
423
424 savefig(p_peak_comparison, plotsdir("sir_des_analysis_peak_comparison.png"))
425
426 # График 5. Сравнение итогового размера эпидемии.
427 p_final_size = bar(
428     x_positions,
429     df_comparison.final_size,
430     label = "Final size",
431     xlabel = "Model",
432     ylabel = "Final epidemic size",
433     title = "SIR: final epidemic size comparison",
434     xticks = (x_positions, df_comparison.model),
435     xrotation = 25,

```

```

436 )
437
438 savefig(p_final_size, plotsdir("sir_des_analysis_final_size_comparison.png"))
439
440 # График 6. Производительность при разных размерах популяции.
441 p_performance = plot(
442     df_performance.population_size,
443     df_performance.elapsed_time,
444     marker = :circle,
445     label = "Elapsed time",
446     xlabel = "Population size",
447     ylabel = "Elapsed time, seconds",
448     title = "SIR DES: performance by population size",
449     linewidth = 2,
450 )
451
452 savefig(p_performance, plotsdir("sir_des_analysis_performance.png"))
453
454 println()
455 println("SIR DES analysis completed.")
456 println("Saved data:")
457 println("  data/sir_des_analysis_des.csv")
458 println("  data/sir_des_analysis_ode.csv")
459 println("  data/sir_des_analysis_vaccination.csv")
460 println("  data/sir_des_analysis_fixed_recovery.csv")
461 println("  data/sir_des_analysis_fixed_recovery_events.csv")
462 println("  data/sir_des_analysis_comparison.csv")
463 println("  data/sir_des_analysis_performance.csv")

```

```

464 println("Saved plots:")
465 println("  plots/sir_des_analysis_des_vs_ode.png")
466 println("  plots/sir_des_analysis_vaccination.png")
467 println("  plots/sir_des_analysis_fixed_recovery.png")
468 println("  plots/sir_des_analysis_peak_comparison.png")
469 println("  plots/sir_des_analysis_final_size_comparison.png")
470 println("  plots/sir_des_analysis_performance.png")

```

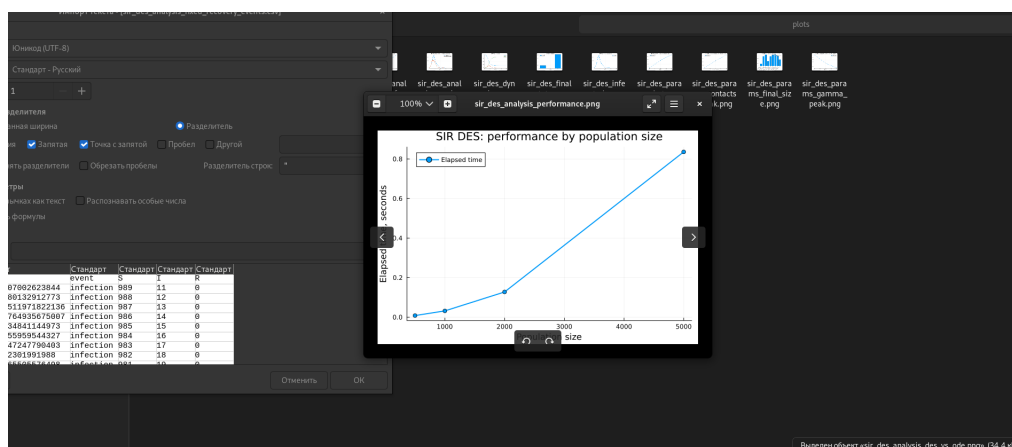


Рисунок 8.1: Выполнение и результаты аналитического скрипта

8.2 Сравнение DES и ODE

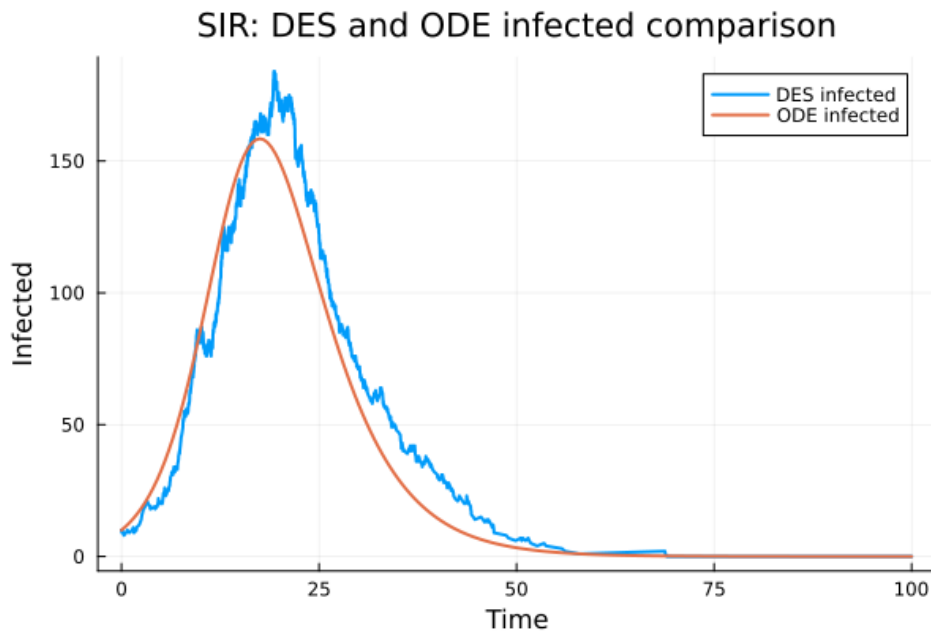


Рисунок 8.2: SIR: DES and ODE infected comparison

График показывает, что DES и ODE дают похожую форму эпидемической волны: число инфицированных сначала растёт, затем достигает пика и постепенно снижается. При этом DES-кривая имеет случайные колебания, так как события заражения и выздоровления происходят стохастически. ODE-кривая является гладкой, поскольку описывает усреднённую непрерывную динамику.

8.3 Сценарий вакцинации

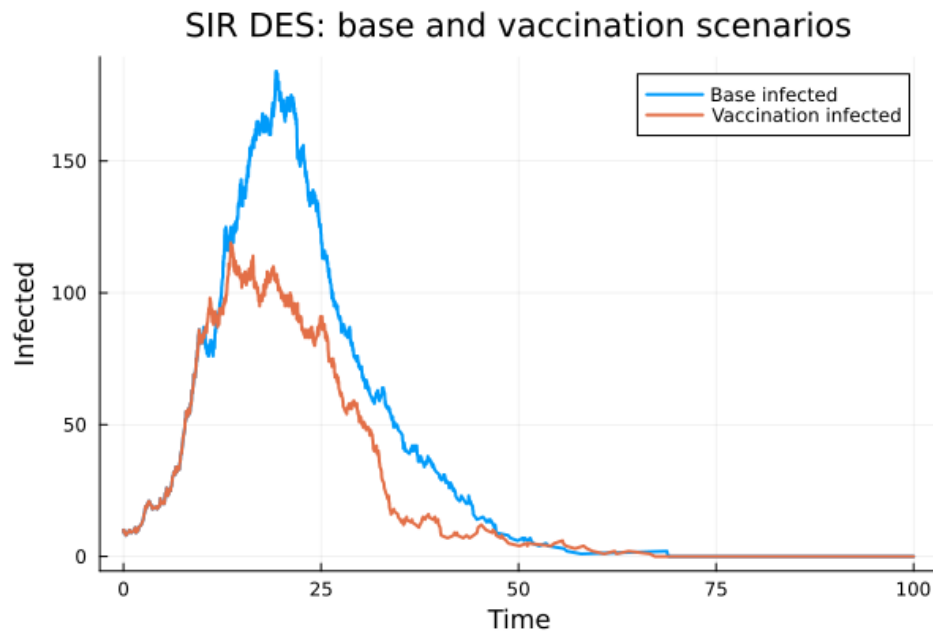


Рисунок 8.3: SIR DES: base and vaccination scenarios

На графике сравниваются базовый DES-сценарий и сценарий вакцинации. В сценарии вакцинации часть восприимчивых агентов переводится в состояние R, то есть исключается из дальнейшего процесса заражения. Поэтому пик инфицированных становится ниже, а эпидемическая волна менее выраженной.

8.4 Фиксированная длительность болезни

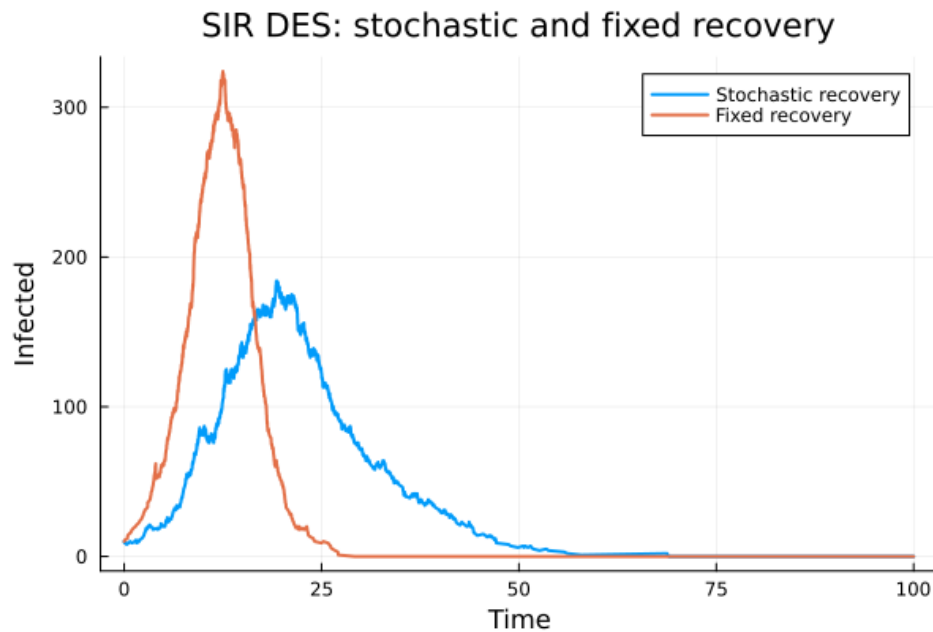


Рисунок 8.4: SIR DES: stochastic and fixed recovery

На этом графике сравниваются стохастическое выздоровление и фиксированная длительность болезни. В варианте с фиксированным временем болезни пик получается более резким и высоким, так как агенты остаются инфицированными одинаковое время. Это показывает, что распределение времени болезни влияет на форму эпидемической кривой.

8.5 Сравнение пиков инфекции

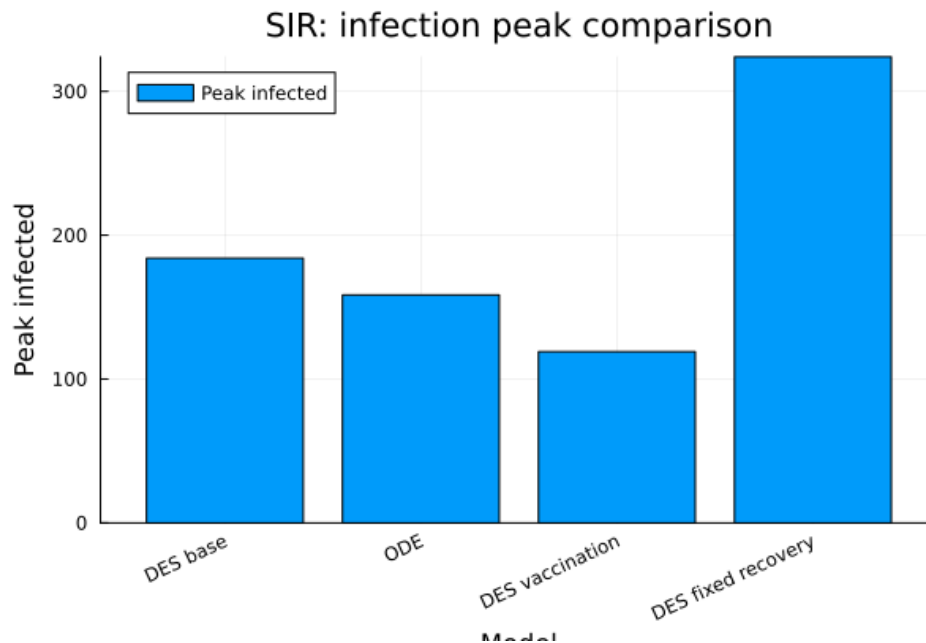


Рисунок 8.5: SIR: infection peak comparison

Диаграмма показывает максимальное число инфицированных для разных вариантов модели. Самый низкий пик наблюдается в сценарии вакцинации, так как часть восприимчивых агентов заранее исключается из процесса распространения инфекции. Самый высокий пик возникает в модели с фиксированным временем выздоровления.

8.6 Сравнение итогового размера эпидемии

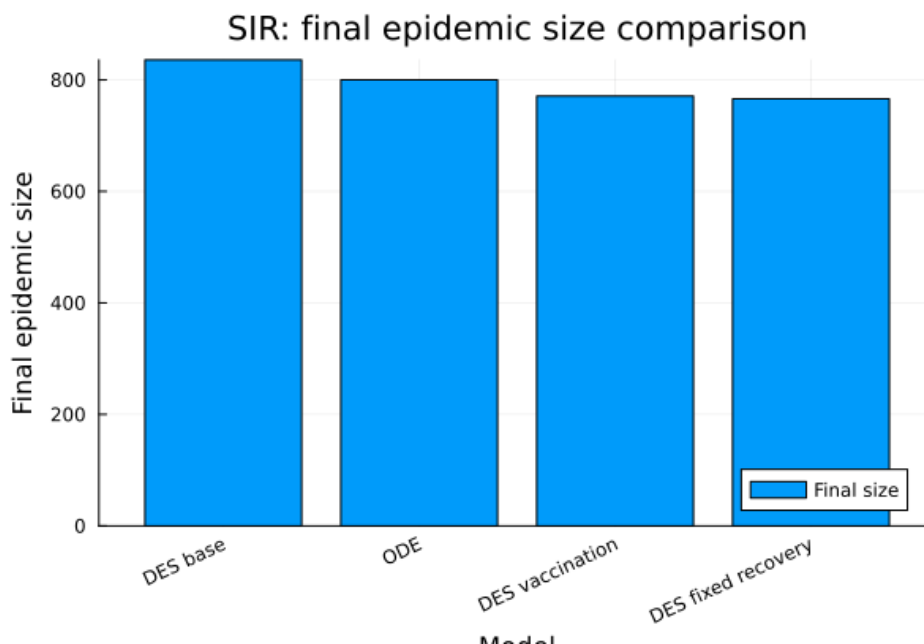


Рисунок 8.6: SIR: final epidemic size comparison

На графике итогового размера эпидемии видно, что все сценарии приводят к значительному числу агентов в состоянии R. Сценарий вакцинации снижает итоговый размер эпидемии по сравнению с базовым DES-сценарием. Различия по итоговому размеру меньше, чем различия по пику инфекции, что означает: форма волны может сильно меняться даже при близком итоговом числе затронутых агентов.

8.7 Производительность

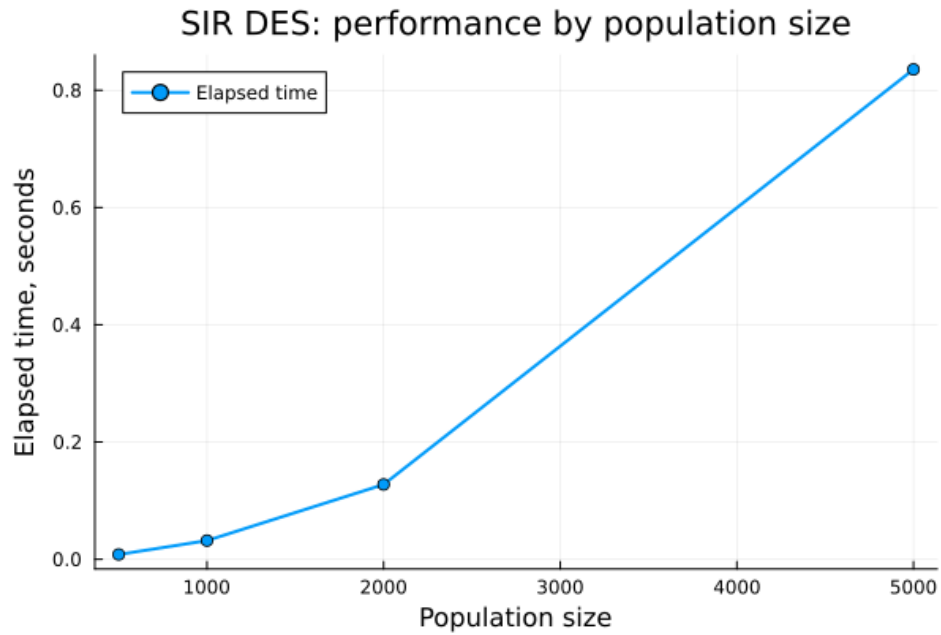


Рисунок 8.7: SIR DES: performance by population size

График показывает зависимость времени выполнения модели от размера популяции. При малом числе агентов модель выполняется почти мгновенно. При увеличении популяции время выполнения возрастает, поскольку необходимо обрабатывать больше событий заражения и выздоровления.

9 Literate-версия аналитического скрипта

Для аналитического скрипта была подготовлена literate-версия `scripts/sir_des_analysis_literate`. Она сохраняет вычислительную логику обычного аналитического скрипта, но дополнена пояснениями для дальнейшей генерации notebook и Quarto-документа.

```
1 # # Лабораторная работа №8
2 #
3 # ## Аналитический скрипт для DES SIR-модели
4 #
5 # В данном скрипте выполняется расширенный анализ дискретно-событийной
6 # SIR-модели.
7 #
8 # Здесь рассматриваются:
9 #
10 # - сравнение DES-модели с детерминированной SIR-моделью на ОДУ;
11 # - сценарий вакцинации;
12 # - вариант с фиксированной длительностью болезни;
13 # - сравнение пиков инфекции;
14 # - сравнение итогового размера эпидемии;
15 # - оценка производительности при разных размерах популяции.
16 #
```

```

17 # Базовая модель уже была реализована в `src/sir_model.jl`,
18 # поэтому здесь мы подключаем готовый source-модуль и используем его функции.
19
20 using DrWatson
21 @quickactivate "project"
22
23 include(scrdir("sir_model.jl"))
24 using .SIRModel
25
26 using CSV
27 using DataFrames
28 using Plots
29 using DifferentialEquations
30 using Random
31 using Distributions
32 using Literate
33
34 # ## Базовые параметры
35 #
36 # Зададим параметры, которые будут использоваться для сравнения разных
37 # вариантов модели. Эти параметры совпадают с базовым запуском SIR DES.
38
39 params = SIRParameters(
40     990,      # начальное число восприимчивых
41     10,       # начальное число инфицированных
42     0,        # начальное число выздоровевших
43     0.05,     # вероятность передачи инфекции
44     10.0,     # среднее число контактов

```

```

45     0.25,      # интенсивность выздоровления
46     100.0,    # максимальное время моделирования
47     123,      # seed
48 )
49
50 # ## Базовый DES-запуск
51 #
52 # Сначала выполним базовый запуск дискретно-событийной SIR-модели.
53 # Его результаты будут использоваться как основной сценарий для сравнения.
54
55 des_result = simulate_sir_des(params)
56 df_des = result_dataframe(des_result)
57 df_des_summary = summary_dataframe(des_result)
58
59 # ## Детерминированная SIR-модель
60 #
61 # Для сравнения реализуем классическую SIR-модель в виде системы
62 # дифференциальных уравнений.
63 #
64 # В отличие от DES-подхода, здесь состояние системы меняется непрерывно,
65 # а результат представляет собой гладкую траекторию.
66
67 function sir_ode!(du, u, p, t)
68     beta, c, gamma, N = p
69
70     S = u[1]
71     I = u[2]
72     R = u[3]

```

```

73
74     infection = beta * c * S * I / N
75     recovery = gamma * I
76
77     du[1] = -infection
78     du[2] = infection - recovery
79     du[3] = recovery
80
81     return nothing
82 end
83
84 N = params.S0 + params.I0 + params.R0
85 u0 = [params.S0, params.I0, params.R0]
86 p = [params.beta, params.c, params.gamma, N]
87 tspan = (0.0, params.tmax)
88
89 ode_problem = ODEProblem(sir_ode!, u0, tspan, p)
90 ode_solution = solve(ode_problem, Tsit5(); saveat = 0.5)
91
92 ode_values = Array(ode_solution)
93
94 df_ode = DataFrame(
95     t = Float64.(ode_solution.t),
96     S = ode_values[1, :],
97     I = ode_values[2, :],
98     R = ode_values[3, :],
99 )
100

```

```

101 ode_peak_I, ode_peak_index = findmax(df_ode.I)
102 ode_peak_time = df_ode.t[ode_peak_index]
103
104 df_ode_summary = DataFrame(
105     model = ["ODE"],
106     peak_I = [ode_peak_I],
107     peak_time = [ode_peak_time],
108     final_S = [df_ode.S[end]],
109     final_I = [df_ode.I[end]],
110     final_R = [df_ode.R[end]],
111     final_size = [df_ode.R[end] - params.R0],
112 )
113
114 # ## Сценарий вакцинации
115 #
116 # Далее рассмотрим дополнительный сценарий вакцинации.
117 #
118 # В заданный момент времени часть восприимчивых агентов переводится
119 # в состояние `R`. Это означает, что они больше не участвуют в процессе
120 # заражения.
121
122 vaccination_result = simulate_sir_des_vaccination(
123     params;
124     vaccination_time = 10.0,
125     vaccination_fraction = 0.25,
126 )
127
128 df_vaccination = result_dataframe(vaccination_result)

```

```

129 df_vaccination_summary = summary_dataframe(vaccination_result)
130
131 # ## Модель с фиксированной длительностью болезни
132 #
133 # Следующее дополнительное задание связано с заменой случайного времени
134 # выздоровления на фиксированную длительность болезни.
135 #
136 # В базовой DES-модели время выздоровления фактически является случайным,
137 # а здесь каждый инфицированный остаётся в состоянии `I` ровно
138 # `1 / gamma` единиц времени.
139
140 function simulate_sir_des_fixed_recovery(
141     params::SIRParameters;
142     illness_duration::Float64 = 1.0 / params.gamma,
143 )
144     rng = MersenneTwister(params.seed)
145
146     S = params.S0
147     I = params.I0
148     R = params.R0
149
150     N = S + I + R
151     t = 0.0
152
153     time = Float64[t]
154     S_values = Int[S]
155     I_values = Int[I]
156     R_values = Int[R]

```

```

157
158 events = DataFrame(
159     t = Float64[],
160     event = Symbol[],
161     S = Int[],
162     I = Int[],
163     R = Int[],
164 )
165
166 # Для начальных инфицированных заранее задаётся момент выздоровления.
167 recovery_times = fill(illness_duration, I)
168
169 while t < params.tmax
170     if I == 0 && isempty(recovery_times)
171         break
172     end
173
174     infection_rate = params.beta * params.c * S * I / N
175
176     if infection_rate > 0.0 && S > 0
177         next_infection_t = t + rand(rng, Exponential(1.0 / infection_rate))
178     else
179         next_infection_t = Inf
180     end
181
182     if !isempty(recovery_times)
183         next_recovery_t, recovery_index = findmin(recovery_times)
184     else

```



```

185         next_recovery_t = Inf
186         recovery_index = 0
187     end
188
189     next_t = min(next_infection_t, next_recovery_t)
190
191     if next_t == Inf || next_t > params.tmax
192         break
193     end
194
195     t = next_t
196
197     if next_recovery_t <= next_infection_t
198         I -= 1
199         R += 1
200         deleteat!(recovery_times, recovery_index)
201
202         push!(
203             events,
204             (
205                 t = t,
206                 event = :fixed_recovery,
207                 S = S,
208                 I = I,
209                 R = R,
210             ),
211         )
212     else

```

```

213         S -= 1
214         I += 1
215         push!(recovery_times, t + illness_duration)
216
217         push!(
218             events,
219             (
220                 t = t,
221                 event = :infection,
222                 S = S,
223                 I = I,
224                 R = R,
225             ),
226         )
227     end
228
229     push!(time, t)
230     push!(S_values, S)
231     push!(I_values, I)
232     push!(R_values, R)
233 end
234
235 if time[end] < params.tmax
236     push!(time, params.tmax)
237     push!(S_values, S)
238     push!(I_values, I)
239     push!(R_values, R)
240 end

```

```

241
242     df_result = DataFrame(
243         t = time,
244         S = S_values,
245         I = I_values,
246         R = R_values,
247     )
248
249     peak_I, peak_index = findmax(df_result.I)
250     peak_time = df_result.t[peak_index]
251
252     df_summary = DataFrame(
253         model = ["DES fixed recovery"],
254         peak_I = [Float64(peak_I)],
255         peak_time = [Float64(peak_time)],
256         final_S = [Float64(df_result.S[end])],
257         final_I = [Float64(df_result.I[end])],
258         final_R = [Float64(df_result.R[end])],
259         final_size = [Float64(df_result.R[end] - params.R0)],
260         events_count = [nrow(events)],
261         illness_duration = [illness_duration],
262     )
263
264     return df_result, events, df_summary
265 end
266
267 df_fixed, df_fixed_events, df_fixed_summary = simulate_sir_des_fixed_recovery(
268     params;

```

```

269     illness_duration = 1.0 / params.gamma,
270 )
271
272 # ## Общая таблица сравнения
273 #
274 # Соберём в одну таблицу основные показатели для всех вариантов:
275 #
276 # - базовая DES-модель;
277 # - детерминированная ODE-модель;
278 # - DES-модель с вакцинацией;
279 # - DES-модель с фиксированной длительностью болезни.
280
281 df_des_summary.model .= "DES base"
282 df_vaccination_summary.model .= "DES vaccination"
283
284 df_comparison = DataFrame(
285     model = String[],
286     peak_I = Float64[],
287     peak_time = Float64[],
288     final_S = Float64[],
289     final_I = Float64[],
290     final_R = Float64[],
291     final_size = Float64[],
292 )
293
294 push!(
295     df_comparison,
296     (

```

```

297     model = "DES base",
298     peak_I = Float64(df_des_summary.peak_I[1]),
299     peak_time = Float64(df_des_summary.peak_time[1]),
300     final_S = Float64(df_des_summary.final_S[1]),
301     final_I = Float64(df_des_summary.final_I[1]),
302     final_R = Float64(df_des_summary.final_R[1]),
303     final_size = Float64(df_des_summary.final_size[1]),
304 ),
305 )
306
307 push!(
308     df_comparison,
309     (
310         model = "ODE",
311         peak_I = Float64(df_ode_summary.peak_I[1]),
312         peak_time = Float64(df_ode_summary.peak_time[1]),
313         final_S = Float64(df_ode_summary.final_S[1]),
314         final_I = Float64(df_ode_summary.final_I[1]),
315         final_R = Float64(df_ode_summary.final_R[1]),
316         final_size = Float64(df_ode_summary.final_size[1]),
317     ),
318 )
319
320 push!(
321     df_comparison,
322     (
323         model = "DES vaccination",
324         peak_I = Float64(df_vaccination_summary.peak_I[1]),

```

```

325     peak_time = Float64(df_vaccination_summary.peak_time[1]),
326     final_S = Float64(df_vaccination_summary.final_S[1]),
327     final_I = Float64(df_vaccination_summary.final_I[1]),
328     final_R = Float64(df_vaccination_summary.final_R[1]),
329     final_size = Float64(df_vaccination_summary.final_size[1]),
330 ),
331 )
332
333 push!(
334     df_comparison,
335     (
336         model = "DES fixed recovery",
337         peak_I = Float64(df_fixed_summary.peak_I[1]),
338         peak_time = Float64(df_fixed_summary.peak_time[1]),
339         final_S = Float64(df_fixed_summary.final_S[1]),
340         final_I = Float64(df_fixed_summary.final_I[1]),
341         final_R = Float64(df_fixed_summary.final_R[1]),
342         final_size = Float64(df_fixed_summary.final_size[1]),
343     ),
344 )
345
346 # ## Оценка производительности
347 #
348 # Для оценки производительности выполним запуск DES-модели при разных
349 # размерах популяции. Для каждого размера измеряется время выполнения.
350
351 population_sizes = [500, 1000, 2000, 5000]
352

```

```

353 df_performance = DataFrame(
354     population_size = Int[],
355     elapsed_time = Float64[],
356     peak_I = Int[],
357     final_size = Int[],
358     events_count = Int[],
359 )
360
361 for population_size in population_sizes
362     test_params = SIRParameters(
363         population_size - 10,
364         10,
365         0,
366         params.beta,
367         params.c,
368         params.gamma,
369         params.tmax,
370         5000 + population_size,
371     )
372
373     test_result = nothing
374
375     elapsed_time = @elapsed begin
376         test_result = simulate_sir_des(test_params)
377     end
378
379     test_summary = summary_dataframe(test_result)
380

```

```

381     push!(
382         df_performance,
383         (
384             population_size = population_size,
385             elapsed_time = elapsed_time,
386             peak_I = test_summary.peak_I[1],
387             final_size = test_summary.final_size[1],
388             events_count = test_summary.events_count[1],
389         ),
390     )
391 end
392
393 # ## Сохранение таблиц
394 #
395 # Сохраним все полученные таблицы в каталог `data/`.
396
397 CSV.write(datadir("sir_des_analysis_literate_des.csv"), df_des)
398 CSV.write(datadir("sir_des_analysis_literate_ode.csv"), df_ode)
399 CSV.write(datadir("sir_des_analysis_literate_vaccination.csv"), df_vaccination)
400 CSV.write(datadir("sir_des_analysis_literate_fixed_recovery.csv"), df_fixed)
401 CSV.write(datadir("sir_des_analysis_literate_fixed_recovery_events.csv"), df_fixed_ev)
402 CSV.write(datadir("sir_des_analysis_literate_comparison.csv"), df_comparison)
403 CSV.write(datadir("sir_des_analysis_literate_performance.csv"), df_performance)
404
405 println("SIR DES analysis comparison:")
406 println(df_comparison)
407
408 println()

```



```

409 println("SIR DES performance:")
410 println(df_performance)
411
412 # ## Сравнение DES и ODE
413 #
414 # Первый график сравнивает динамику инфицированных в дискретно-событийной
415 # и детерминированной моделях.
416
417 p_compare_infected = plot(
418     df_des.t,
419     df_des.I,
420     label = "DES infected",
421     xlabel = "Time",
422     ylabel = "Infected",
423     title = "SIR: DES and ODE infected comparison",
424     linewidth = 2,
425 )
426
427 plot!(
428     p_compare_infected,
429     df_ode.t,
430     df_ode.I,
431     label = "ODE infected",
432     linewidth = 2,
433 )
434
435 savefig(p_compare_infected, plotsdir("sir_des_analysis_literate_des_vs_ode.png"))
436

```

```

437 # ## Сценарий вакцинации
438 #
439 # Второй график показывает, как вакцинация влияет на число инфицированных.
440 # Вакцинация снижает количество восприимчивых агентов, поэтому пик инфекции
441 # становится ниже.
442
443 p_vaccination = plot(
444     df_des.t,
445     df_des.I,
446     label = "Base infected",
447     xlabel = "Time",
448     ylabel = "Infected",
449     title = "SIR DES: base and vaccination scenarios",
450     linewidth = 2,
451 )
452
453 plot!(
454     p_vaccination,
455     df_vaccination.t,
456     df_vaccination.I,
457     label = "Vaccination infected",
458     linewidth = 2,
459 )
460
461 savefig(p_vaccination, plotsdir("sir_des_analysis_literate_vaccination.png"))
462
463 # ## Фиксированная длительность болезни
464 #

```

```

465 # Третий график сравнивает стохастическое выздоровление и фиксированную
466 # длительность болезни.
467
468 p_fixed = plot(
469     df_des.t,
470     df_des.I,
471     label = "Stochastic recovery",
472     xlabel = "Time",
473     ylabel = "Infected",
474     title = "SIR DES: stochastic and fixed recovery",
475     linewidth = 2,
476 )
477
478 plot!(
479     p_fixed,
480     df_fixed.t,
481     df_fixed.I,
482     label = "Fixed recovery",
483     linewidth = 2,
484 )
485
486 savefig(p_fixed, plotsdir("sir_des_analysis_literate_fixed_recovery.png"))
487
488 # ## Сравнение пиков инфекции
489 #
490 # На следующей диаграмме сравним максимальное количество инфицированных
491 # для всех вариантов модели.
492

```

```

493 x_positions = collect(1:nrow(df_comparison))
494
495 p_peak_comparison = bar(
496     x_positions,
497     df_comparison.peak_I,
498     label = "Peak infected",
499     xlabel = "Model",
500     ylabel = "Peak infected",
501     title = "SIR: infection peak comparison",
502     xticks = (x_positions, df_comparison.model),
503     xrotation = 25,
504 )
505
506 savefig(p_peak_comparison, plotsdir("sir_des_analysis_iterate_peak_comparison.png"))
507
508 # ## Сравнение итогового размера эпидемии
509 #
510 # Итоговый размер эпидемии показывает, сколько агентов перешло в состояние
511 # `R` к концу моделирования.
512
513 p_final_size = bar(
514     x_positions,
515     df_comparison.final_size,
516     label = "Final size",
517     xlabel = "Model",
518     ylabel = "Final epidemic size",
519     title = "SIR: final epidemic size comparison",
520     xticks = (x_positions, df_comparison.model),

```

```

521     xrotation = 25,
522 )
523
524 savefig(p_final_size, plotsdir("sir_des_analysis_literate_final_size_comparison.png"))
525
526 # ## Производительность
527 #
528 # Последний график показывает зависимость времени выполнения модели
529 # от размера популяции.
530
531 p_performance = plot(
532     df_performance.population_size,
533     df_performance.elapsed_time,
534     marker = :circle,
535     label = "Elapsed time",
536     xlabel = "Population size",
537     ylabel = "Elapsed time, seconds",
538     title = "SIR DES: performance by population size",
539     linewidth = 2,
540 )
541
542 savefig(p_performance, plotsdir("sir_des_analysis_literate_performance.png"))
543
544 # ## Итог
545 #
546 # В результате выполнения аналитического скрипта были получены таблицы
547 # сравнения, графики разных сценариев и оценка производительности модели.
548

```

```

549 println()
550 println("SIR DES analysis literate completed.")
551 println("Saved data:")
552 println("  data/sir_des_analysis_literate_des.csv")
553 println("  data/sir_des_analysis_literate_ode.csv")
554 println("  data/sir_des_analysis_literate_vaccination.csv")
555 println("  data/sir_des_analysis_literate_fixed_recovery.csv")
556 println("  data/sir_des_analysis_literate_fixed_recovery_events.csv")
557 println("  data/sir_des_analysis_literate_comparison.csv")
558 println("  data/sir_des_analysis_literate_performance.csv")
559 println("Saved plots:")
560 println("  plots/sir_des_analysis_literate_des_vs_ode.png")
561 println("  plots/sir_des_analysis_literate_vaccination.png")
562 println("  plots/sir_des_analysis_literate_fixed_recovery.png")
563 println("  plots/sir_des_analysis_literate_peak_comparison.png")
564 println("  plots/sir_des_analysis_literate_final_size_comparison.png")
565 println("  plots/sir_des_analysis_literate_performance.png")
566
567 # ## Генерация файлов из literate-версии
568 #
569 # В конце создаются производные файлы:
570 #
571 # - чистый Julia-скрипт;
572 # - Jupyter Notebook;
573 # - Quarto-документ.
574
575 output_dir = scriptsdir("generated", "sir_des_analysis_literate")
576 mkpath(output_dir)

```

```

577
578 Literate.script(@__FILE__, output_dir)
579 Literate.notebook(@__FILE__, output_dir)
580 Literate.markdown(@__FILE__, output_dir; flavor = Literate.QuartoFlavor())
581
582 println()
583 println("Generated literate outputs:")
584 println("  scripts/generated/sir_des_analysis_literate/")

```

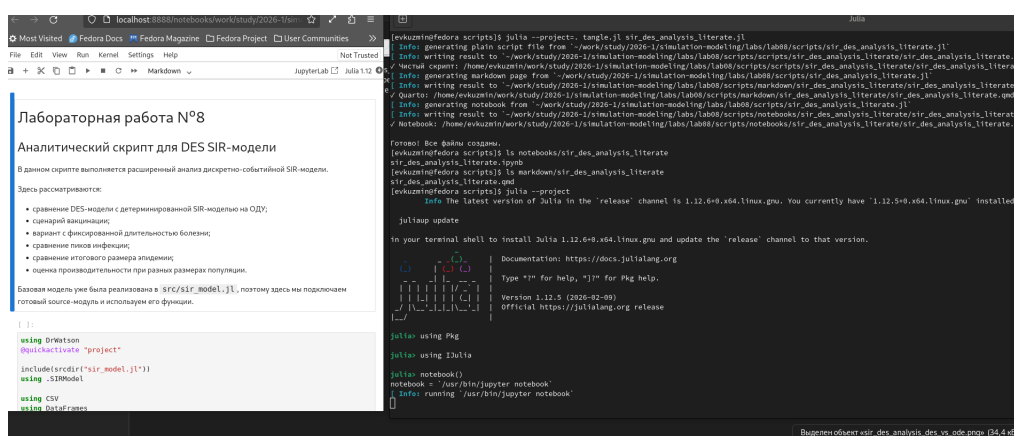


Рисунок 9.1: Создание и notebook literate-версии второго скрипта

10 Параметризованная аналитическая версия

10.1 Скрипт `sir_des_analysis_params.jl`

Параметризованная аналитическая версия используется для исследования дополнительных сценариев: вакцинации, фиксированной длительности болезни, демографии, SEIR-расширения и производительности.

```
1 # # Лабораторная работа №8
2 #
3 # ## Параметризованная аналитическая версия DES SIR-модели
4 #
5 # В данном скрипте выполняется расширенный параметризованный анализ.
6 #
7 # Скрипт закрывает дополнительные сценарии:
8 #
9 # - вакцинация;
10 # - фиксированная длительность болезни;
11 # - оценка производительности;
12 # - демографический сценарий;
13 # - SEIR-расширение.
14 #
```



```

15 # Анализ чувствительности по `beta`, `c`, `gamma` уже был выполнен
16 # в файле `sir_des_params.jl`.
17
18 using DrWatson
19 @quickactivate "project"
20
21 include(scrdir("sir_model.jl"))
22 using .SIRModel
23
24 using CSV
25 using DataFrames
26 using Plots
27 using Random
28 using Distributions
29
30 # ## Базовые параметры
31 #
32 # Зададим базовые параметры модели. Они используются как основа для
33 # всех дополнительных сценариев.
34
35 base_params = SIRParameters(
36     990,      # S0
37     10,       # I0
38     0,        # R0
39     0.05,     # beta
40     10.0,     # c
41     0.25,     # gamma
42     100.0,    # tmax

```

```

43     123,      # seed
44 )
45
46 # ## Вспомогательная функция для краткой сводки
47 #
48 # Эта функция используется для сценариев, где результат представлен
49 # таблицей с колонками `t`, `S`, `I`, `R`.
50
51 function simple_summary(df::DataFrame, model_name::String)
52     peak_I, peak_index = findmax(df.I)
53     peak_time = df.t[peak_index]
54
55     return DataFrame(
56         model = [model_name],
57         peak_I = [Float64(peak_I)],
58         peak_time = [Float64(peak_time)],
59         final_S = [Float64(df.S[end])],
60         final_I = [Float64(df.I[end])],
61         final_R = [Float64(df.R[end])],
62         final_size = [Float64(df.R[end])],
63     )
64 end
65
66 # ## Модель с фиксированной длительностью болезни
67 #
68 # В этом варианте время выздоровления не случайное, а фиксированное.
69 # Каждый инфицированный остаётся в состоянии `I` ровно `illness_duration`
70 # единиц времени.

```

```

71
72 function simulate_fixed_recovery(
73     params::SIRParameters;
74     illness_duration::Float64 = 1.0 / params.gamma,
75 )
76     rng = MersenneTwister(params.seed)
77
78     S = params.S0
79     I = params.I0
80     R = params.R0
81
82     N = S + I + R
83     t = 0.0
84
85     time = Float64[t]
86     S_values = Int[S]
87     I_values = Int[I]
88     R_values = Int[R]
89
90     recovery_times = fill(illness_duration, I)
91
92     while t < params.tmax
93         if I == 0 && isempty(recovery_times)
94             break
95         end
96
97         infection_rate = params.beta * params.c * S * I / N
98

```

```

99     if infection_rate > 0.0 && S > 0
100         next_infection_t = t + rand(rng, Exponential(1.0 / infection_rate))
101     else
102         next_infection_t = Inf
103     end
104
105     if !isempty(recovery_times)
106         next_recovery_t, recovery_index = findmin(recovery_times)
107     else
108         next_recovery_t = Inf
109         recovery_index = 0
110     end
111
112     next_t = min(next_infection_t, next_recovery_t)
113
114     if next_t == Inf || next_t > params.tmax
115         break
116     end
117
118     t = next_t
119
120     if next_recovery_t <= next_infection_t
121         I -= 1
122         R += 1
123         deleteat!(recovery_times, recovery_index)
124     else
125         S -= 1
126         I += 1

```

```

127         push!(recovery_times, t + illness_duration)
128     end
129
130     push!(time, t)
131     push!(S_values, S)
132     push!(I_values, I)
133     push!(R_values, R)
134 end
135
136 if time[end] < params.tmax
137     push!(time, params.tmax)
138     push!(S_values, S)
139     push!(I_values, I)
140     push!(R_values, R)
141 end
142
143 return DataFrame(
144     t = time,
145     S = S_values,
146     I = I_values,
147     R = R_values,
148 )
149 end
150
151 # ## Демографический сценарий
152 #
153 # В этом варианте в модель добавлены простые демографические события:
154 # рождение и смерть.

```

```

155 #
156 # Рождение увеличивает число восприимчивых `S`, а смерть может уменьшать
157 # одну из групп `S`, `I` или `R`.
158
159 function simulate_sir_demography(
160     params::SIRParameters;
161     mu::Float64 = 0.005,
162 )
163     rng = MersenneTwister(params.seed)
164
165     S = params.S0
166     I = params.I0
167     R = params.R0
168
169     t = 0.0
170
171     time = Float64[t]
172     S_values = Int[S]
173     I_values = Int[I]
174     R_values = Int[R]
175
176     while t < params.tmax
177         N = S + I + R
178
179         if N <= 0
180             break
181         end
182

```

```

183     infection_rate = params.beta * params.c * S * I / N
184     recovery_rate = params.gamma * I
185     birth_rate = mu * N
186     death_S_rate = mu * S
187     death_I_rate = mu * I
188     death_R_rate = mu * R
189
190     total_rate =
191         infection_rate +
192         recovery_rate +
193         birth_rate +
194         death_S_rate +
195         death_I_rate +
196         death_R_rate
197
198     if total_rate <= 0.0
199         break
200     end
201
202     dt = rand(rng, Exponential(1.0 / total_rate))
203     next_t = t + dt
204
205     if next_t > params.tmax
206         break
207     end
208
209     t = next_t
210

```

```

211     r = rand(rng) * total_rate
212
213     if r <= infection_rate && S > 0
214         S -= 1
215         I += 1
216     elseif r <= infection_rate + recovery_rate && I > 0
217         I -= 1
218         R += 1
219     elseif r <= infection_rate + recovery_rate + birth_rate
220         S += 1
221     elseif r <= infection_rate + recovery_rate + birth_rate + death_S_rate && S > 0
222         S -= 1
223     elseif r <= infection_rate + recovery_rate + birth_rate + death_S_rate + death_R_rate
224         I -= 1
225     elseif R > 0
226         R -= 1
227     end
228
229     push!(time, t)
230     push!(S_values, S)
231     push!(I_values, I)
232     push!(R_values, R)
233 end
234
235 if time[end] < params.tmax
236     push!(time, params.tmax)
237     push!(S_values, S)
238     push!(I_values, I)

```



```

239         push!(R_values, R)
240     end
241
242     return DataFrame(
243         t = time,
244         S = S_values,
245         I = I_values,
246         R = R_values,
247     )
248 end
249
250 # ## SEIR-расширение
251 #
252 # В SEIR-модели добавляется состояние `E` – заражённые, но ещё не
253 # инфекционные агенты.
254 #
255 # Переходы:
256 #
257 # - `S` -> `E`;
258 # - `E` -> `I`;
259 # - `I` -> `R`.
260
261 function simulate_seir_des(
262     params::SIRParameters;
263     sigma::Float64 = 0.4,
264 )
265     rng = MersenneTwister(params.seed)
266

```

```

267     S = params.S0
268     E = 0
269     I = params.I0
270     R = params.R0
271
272     N = S + E + I + R
273     t = 0.0
274
275     time = Float64[t]
276     S_values = Int[S]
277     E_values = Int[E]
278     I_values = Int[I]
279     R_values = Int[R]
280
281     while t < params.tmax
282         if E == 0 && I == 0
283             break
284         end
285
286         infection_rate = params.beta * params.c * S * I / N
287         incubation_rate = sigma * E
288         recovery_rate = params.gamma * I
289
290         total_rate = infection_rate + incubation_rate + recovery_rate
291
292         if total_rate <= 0.0
293             break
294         end

```

```

295
296     dt = rand(rng, Exponential(1.0 / total_rate))
297     next_t = t + dt
298
299     if next_t > params.tmax
300         break
301     end
302
303     t = next_t
304
305     r = rand(rng) * total_rate
306
307     if r <= infection_rate && S > 0
308         S -= 1
309         E += 1
310     elseif r <= infection_rate + incubation_rate && E > 0
311         E -= 1
312         I += 1
313     elseif I > 0
314         I -= 1
315         R += 1
316     end
317
318     push!(time, t)
319     push!(S_values, S)
320     push!(E_values, E)
321     push!(I_values, I)
322     push!(R_values, R)

```

```

323     end
324
325     if time[end] < params.tmax
326         push!(time, params.tmax)
327         push!(S_values, S)
328         push!(E_values, E)
329         push!(I_values, I)
330         push!(R_values, R)
331     end
332
333     return DataFrame(
334         t = time,
335         S = S_values,
336         E = E_values,
337         I = I_values,
338         R = R_values,
339     )
340 end
341
342 # ## Анализ сценариев вакцинации
343 #
344 # В этом блоке изменяется доля вакцинированных агентов.
345 # Чем больше эта доля, тем меньше восприимчивых остаётся в системе.
346
347 vaccination_fractions = [0.0, 0.1, 0.25, 0.4, 0.6]
348
349 vaccination_summary = DataFrame(
350     vaccination_fraction = Float64[],

```

```

351     peak_I = Float64[],
352     peak_time = Float64[],
353     final_S = Float64[],
354     final_I = Float64[],
355     final_R = Float64[],
356     final_size = Float64[],
357 )
358
359 for fraction in vaccination_fractions
360     result = simulate_sir_des_vaccination(
361         base_params;
362         vaccination_time = 10.0,
363         vaccination_fraction = fraction,
364     )
365
366     summary = summary_dataframe(result)
367
368     push!(
369         vaccination_summary,
370         (
371             vaccination_fraction = fraction,
372             peak_I = Float64(summary.peak_I[1]),
373             peak_time = Float64(summary.peak_time[1]),
374             final_S = Float64(summary.final_S[1]),
375             final_I = Float64(summary.final_I[1]),
376             final_R = Float64(summary.final_R[1]),
377             final_size = Float64(summary.final_size[1]),
378         ),

```

```

379     )
380 end
381
382 # ## Анализ фиксированной длительности болезни
383 #
384 # Здесь меняется фиксированная длительность болезни.
385 # Это позволяет сравнить, как форма эпидемической волны зависит от
386 # времени нахождения агента в состоянии `I`.
387
388 illness_durations = [2.0, 4.0, 6.0, 8.0]
389
390 fixed_recovery_summary = DataFrame(
391     illness_duration = Float64[],
392     peak_I = Float64[],
393     peak_time = Float64[],
394     final_S = Float64[],
395     final_I = Float64[],
396     final_R = Float64[],
397     final_size = Float64[],
398 )
399
400 for illness_duration in illness_durations
401     df = simulate_fixed_recovery(
402         base_params;
403         illness_duration = illness_duration,
404     )
405
406     summary = simple_summary(df, "fixed_recovery")

```

```

407
408     push!(
409         fixed_recovery_summary,
410         (
411             illness_duration = illness_duration,
412             peak_I = summary.peak_I[1],
413             peak_time = summary.peak_time[1],
414             final_S = summary.final_S[1],
415             final_I = summary.final_I[1],
416             final_R = summary.final_R[1],
417             final_size = summary.final_size[1],
418         ),
419     )
420 end
421
422 # ## Анализ демографического сценария
423 #
424 # В этом блоке изменяется параметр `mu`, который задаёт интенсивность
425 # рождения и смерти.
426
427 demography_mus = [0.0, 0.002, 0.005, 0.01]
428
429 demography_summary = DataFrame(
430     mu = Float64[],
431     peak_I = Float64[],
432     peak_time = Float64[],
433     final_S = Float64[],
434     final_I = Float64[],

```

```

435     final_R = Float64[],
436     final_size = Float64[],
437     final_population = Float64[],
438 )
439
440 for mu in demography_mus
441     df = simulate_sir_demography(base_params; mu = mu)
442     summary = simple_summary(df, "demography")
443
444     push!(
445         demography_summary,
446         (
447             mu = mu,
448             peak_I = summary.peak_I[1],
449             peak_time = summary.peak_time[1],
450             final_S = summary.final_S[1],
451             final_I = summary.final_I[1],
452             final_R = summary.final_R[1],
453             final_size = summary.final_size[1],
454             final_population = Float64(df.S[end] + df.I[end] + df.R[end]),
455         ),
456     )
457 end
458
459 # ## Анализ SEIR-модели
460 #
461 # В этом блоке изменяется параметр `sigma`.
462 # Он отвечает за скорость перехода из состояния `E` в состояние `I`.

```



```

463
464 sigma_values = [0.15, 0.25, 0.5, 1.0]
465
466 seir_summary = DataFrame(
467     sigma = Float64[],
468     peak_E = Float64[],
469     peak_I = Float64[],
470     peak_time_I = Float64[],
471     final_S = Float64[],
472     final_E = Float64[],
473     final_I = Float64[],
474     final_R = Float64[],
475     final_size = Float64[],
476 )
477
478 for sigma in sigma_values
479     df = simulate_seir_des(base_params; sigma = sigma)
480
481     peak_E = maximum(df.E)
482     peak_I, peak_index = findmax(df.I)
483     peak_time_I = df.t[peak_index]
484
485     push!(
486         seir_summary,
487         (
488             sigma = sigma,
489             peak_E = Float64(peak_E),
490             peak_I = Float64(peak_I),

```

```

491         peak_time_I = Float64(peak_time_I),
492         final_S = Float64(df.S[end]),
493         final_E = Float64(df.E[end]),
494         final_I = Float64(df.I[end]),
495         final_R = Float64(df.R[end]),
496         final_size = Float64(df.R[end]),
497     ),
498 )
499 end
500
501 # ## Оценка производительности
502 #
503 # Здесь модель запускается для разных размеров популяции.
504 # Для каждого размера измеряется время выполнения.
505
506 population_sizes = [500, 1000, 2000, 5000]
507
508 performance_summary = DataFrame(
509     population_size = Int[],
510     elapsed_time = Float64[],
511     peak_I = Int[],
512     final_size = Int[],
513     events_count = Int[],
514 )
515
516 for population_size in population_sizes
517     params = SIRParameters(
518         population_size - 10,

```

```

519         10,
520         0,
521         base_params.beta,
522         base_params.c,
523         base_params.gamma,
524         base_params.tmax,
525         7000 + population_size,
526     )
527
528     result = nothing
529
530     elapsed_time = @elapsed begin
531         result = simulate_sir_des(params)
532     end
533
534     summary = summary_dataframe(result)
535
536     push!(
537         performance_summary,
538         (
539             population_size = population_size,
540             elapsed_time = elapsed_time,
541             peak_I = summary.peak_I[1],
542             final_size = summary.final_size[1],
543             events_count = summary.events_count[1],
544         ),
545     )
546 end

```

```
547
548 # ## Сохранение таблиц
549 #
550 # Все сводные таблицы сохраняются в каталог `data/`.
551
552 CSV.write(datadir("sir_des_analysis_params_vaccination.csv"), vaccination_summary)
553 CSV.write(datadir("sir_des_analysis_params_fixed_recovery.csv"), fixed_recovery_summa
554 CSV.write(datadir("sir_des_analysis_params_demography.csv"), demography_summary)
555 CSV.write(datadir("sir_des_analysis_params_seir.csv"), seir_summary)
556 CSV.write(datadir("sir_des_analysis_params_performance.csv"), performance_summary)
557
558 println("Vaccination summary:")
559 println(vaccination_summary)
560
561 println()
562 println("Fixed recovery summary:")
563 println(fixed_recovery_summary)
564
565 println()
566 println("Demography summary:")
567 println(demography_summary)
568
569 println()
570 println("SEIR summary:")
571 println(seir_summary)
572
573 println()
574 println("Performance summary:")
```

```

575 println(performance_summary)
576
577 # ## График влияния доли вакцинации
578 #
579 # На графике сравниваются пик инфекции и итоговый размер эпидемии
580 # при разных долях вакцинации.
581
582 p_vaccination = plot(
583     vaccination_summary.vaccination_fraction,
584     vaccination_summary.peak_I,
585     marker = :circle,
586     label = "Peak I",
587     xlabel = "Vaccination fraction",
588     ylabel = "Value",
589     title = "SIR DES: vaccination parameter scan",
590     linewidth = 2,
591 )
592
593 plot!(
594     p_vaccination,
595     vaccination_summary.vaccination_fraction,
596     vaccination_summary.final_size,
597     marker = :circle,
598     label = "Final size",
599     linewidth = 2,
600 )
601
602 savefig(p_vaccination, plotsdir("sir_des_analysis_params_vaccination.png"))

```

```

603
604 # ## График влияния фиксированной длительности болезни
605 #
606 # Этот график показывает, как увеличение фиксированной длительности болезни
607 # влияет на пик инфекции и итоговый размер эпидемии.
608
609 p_fixed = plot(
610     fixed_recovery_summary.illness_duration,
611     fixed_recovery_summary.peak_I,
612     marker = :circle,
613     label = "Peak I",
614     xlabel = "Illness duration",
615     ylabel = "Value",
616     title = "SIR DES: fixed recovery duration scan",
617     linewidth = 2,
618 )
619
620 plot!(
621     p_fixed,
622     fixed_recovery_summary.illness_duration,
623     fixed_recovery_summary.final_size,
624     marker = :circle,
625     label = "Final size",
626     linewidth = 2,
627 )
628
629 savefig(p_fixed, plotsdir("sir_des_analysis_params_fixed_recovery.png"))
630

```

```

631 # ## График демографического сценария
632 #
633 # На графике показано влияние параметра демографии `mu`
634 # на пик инфекции и финальную численность популяции.
635
636 p_demography = plot(
637     demography_summary.mu,
638     demography_summary.peak_I,
639     marker = :circle,
640     label = "Peak I",
641     xlabel = "Mu",
642     ylabel = "Value",
643     title = "SIR DES: demography parameter scan",
644     linewidth = 2,
645 )
646
647 plot!(
648     p_demography,
649     demography_summary.mu,
650     demography_summary.final_population,
651     marker = :circle,
652     label = "Final population",
653     linewidth = 2,
654 )
655
656 savefig(p_demography, plotsdir("sir_des_analysis_params_demography.png"))
657
658 # ## График SEIR-сценария

```

```

659 #
660 # На графике показано влияние параметра `sigma` на пик скрыто заражённых
661 # и пик инфицированных.
662
663 p_seir = plot(
664     seir_summary.sigma,
665     seir_summary.peak_E,
666     marker = :circle,
667     label = "Peak E",
668     xlabel = "Sigma",
669     ylabel = "Value",
670     title = "SEIR DES: sigma parameter scan",
671     linewidth = 2,
672 )
673
674 plot!(
675     p_seir,
676     seir_summary.sigma,
677     seir_summary.peak_I,
678     marker = :circle,
679     label = "Peak I",
680     linewidth = 2,
681 )
682
683 savefig(p_seir, plotsdir("sir_des_analysis_params_seir.png"))
684
685 # ## График производительности
686 #

```



```

687 # Последний график показывает зависимость времени выполнения от размера
688 # популяции.
689
690 p_performance = plot(
691     performance_summary.population_size,
692     performance_summary.elapsed_time,
693     marker = :circle,
694     label = "Elapsed time",
695     xlabel = "Population size",
696     ylabel = "Elapsed time, seconds",
697     title = "SIR DES: performance parameter scan",
698     linewidth = 2,
699 )
700
701 savefig(p_performance, plotsdir("sir_des_analysis_params_performance.png"))
702
703 # ## Завершение
704 #
705 # В результате выполнения скрипта сохранены таблицы и графики для
706 # параметризованного анализа дополнительных сценариев.
707
708 println()
709 println("SIR DES analysis parameter scan completed.")
710 println("Saved data:")
711 println("  data/sir_des_analysis_params_vaccination.csv")
712 println("  data/sir_des_analysis_params_fixed_recovery.csv")
713 println("  data/sir_des_analysis_params_demography.csv")
714 println("  data/sir_des_analysis_params_seir.csv")

```

```

715 println(" data/sir_des_analysis_params_performance.csv")
716 println("Saved plots:")
717 println(" plots/sir_des_analysis_params_vaccination.png")
718 println(" plots/sir_des_analysis_params_fixed_recovery.png")
719 println(" plots/sir_des_analysis_params_demography.png")
720 println(" plots/sir_des_analysis_params_seir.png")
721 println(" plots/sir_des_analysis_params_performance.png")

```

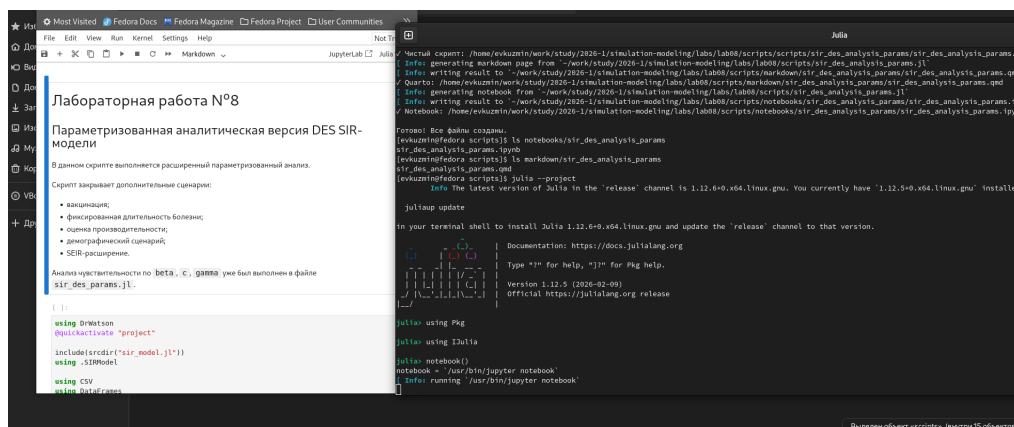


Рисунок 10.1: Создание и notebook параметризованной версии второго скрипта

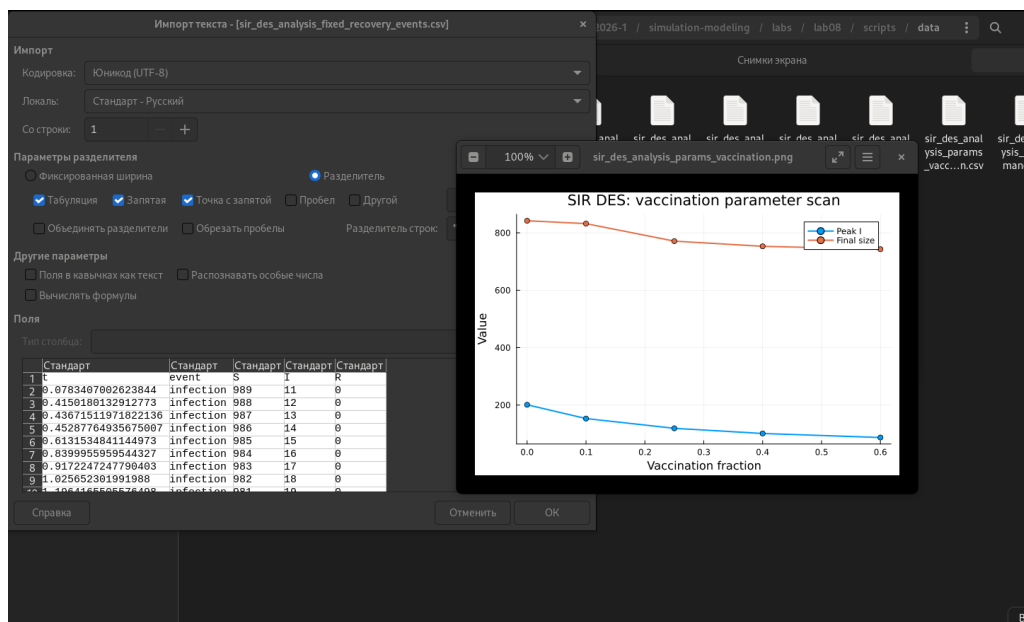


Рисунок 10.2: Результаты параметризованной версии второго скрипта

10.2 Параметризованный сценарий вакцинации

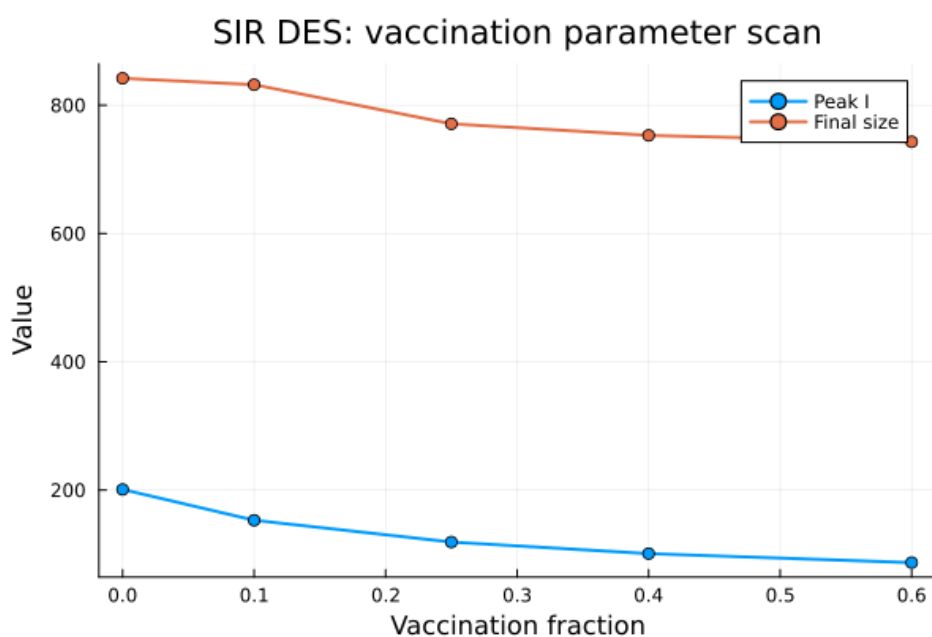


Рисунок 10.3: SIR DES: vaccination parameter scan

В этом сценарии изменяется доля вакцинированных агентов. Чем больше доля вакцинации, тем меньше восприимчивых агентов остаётся в системе, поэтому пик инфицированных и итоговый размер эпидемии снижаются.

10.3 Фиксированная длительность болезни

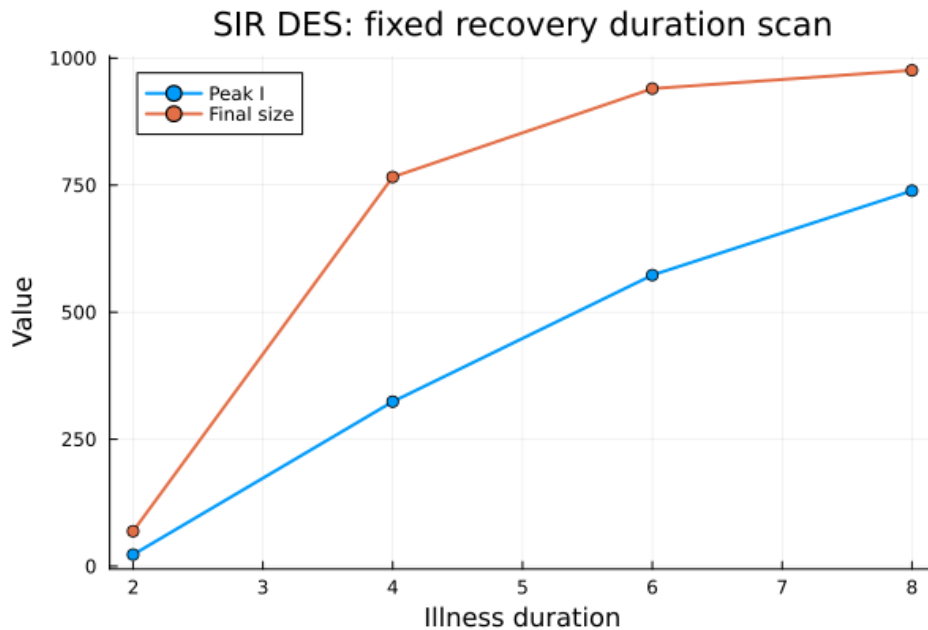


Рисунок 10.4: SIR DES: fixed recovery duration scan

График показывает, что при увеличении фиксированной длительности болезни растёт и пик инфицированных, и итоговый размер эпидемии. Чем дольше агент остаётся инфицированным, тем больше времени он может заражать других.

10.4 Демографический сценарий

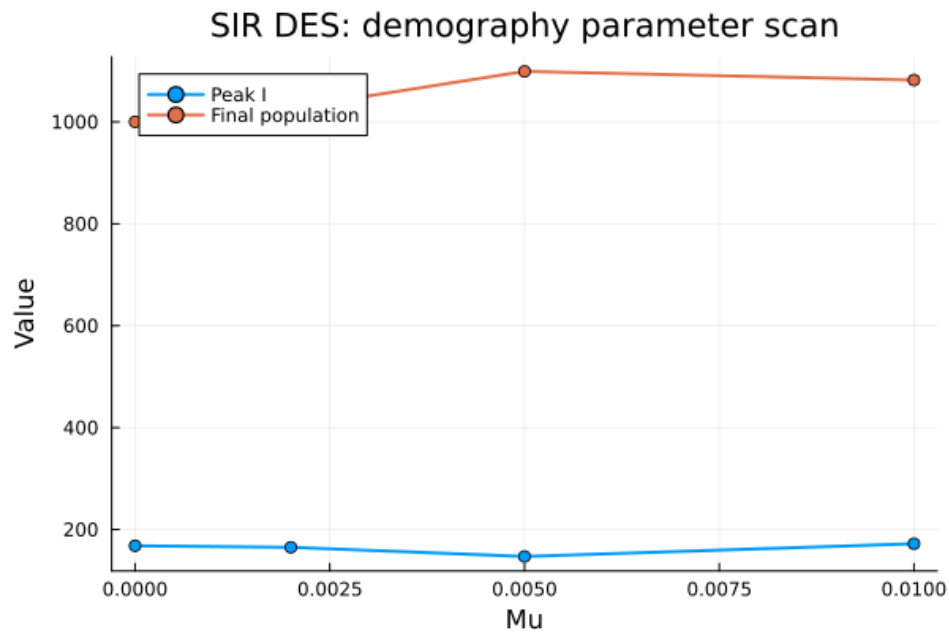


Рисунок 10.5: SIR DES: demography parameter scan

В демографическом сценарии изменяется параметр μ , отвечающий за интенсивность рождений и смертей. При увеличении μ финальная численность популяции может возрасть, поскольку в модель добавляются новые восприимчивые агенты. Пик инфицированных меняется слабее, чем финальная численность.

10.5 SEIR-сценарий

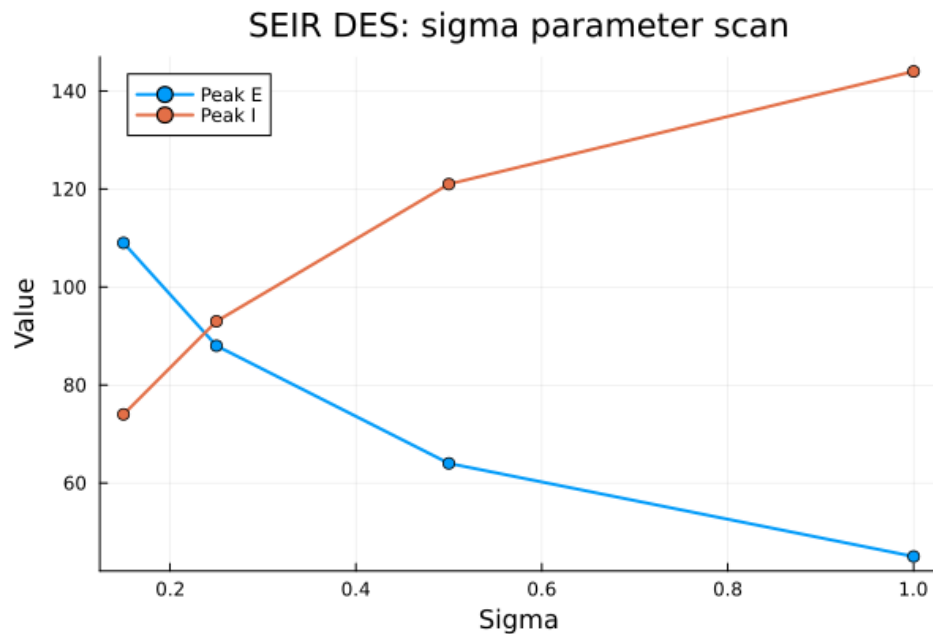


Рисунок 10.6: SEIR DES: sigma parameter scan

В SEIR-модели добавляется промежуточное состояние E — заражённые, но ещё не инфекционные агенты. Параметр σ отвечает за скорость перехода из состояния E в состояние I. При увеличении σ пик скрыто заражённых уменьшается, а пик инфицированных возрастает, так как агенты быстрее становятся инфекционными.

10.6 Производительность в параметризованной версии

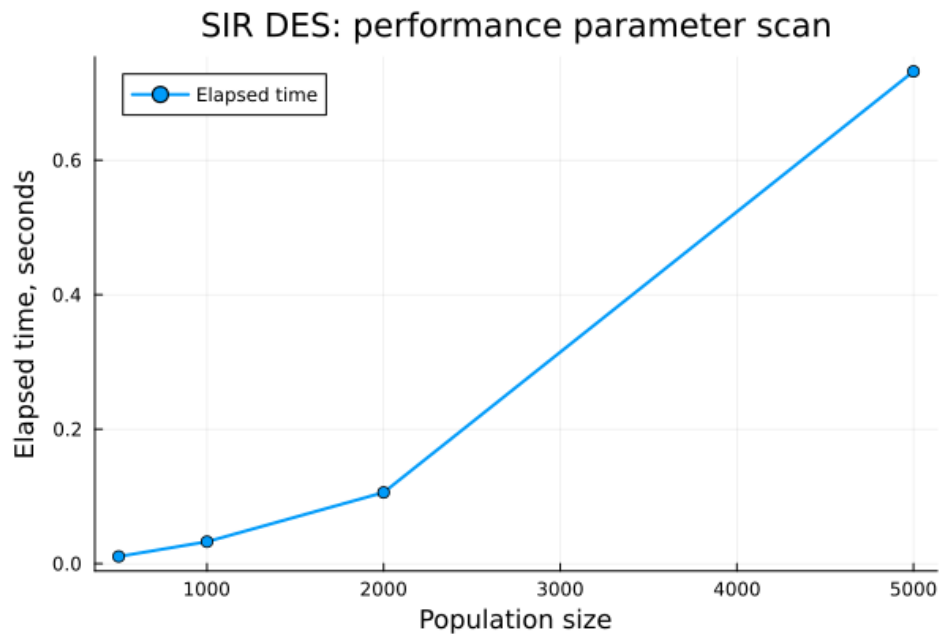


Рисунок 10.7: SIR DES: performance parameter scan

График подтверждает, что при увеличении размера популяции время выполнения возрастает. Это ожидаемо для DES-подхода, так как большее число агентов приводит к увеличению числа событий, которые нужно обработать.

11 Выполнение дополнительных заданий

В рамках лабораторной работы были закрыты дополнительные задания:

Пункт	Реализация
Анализ чувствительности к β , σ , γ	<code>sir_des_params.jl</code>
Детерминированная длительность болезни	<code>sir_des_analysis.jl</code> , <code>sir_des_analysis_params.jl</code>
Оценка производительности	<code>sir_des_analysis.jl</code> , <code>sir_des_analysis_params.jl</code>
Сохранение результатов в CSV	все основные и параметризованные скрипты
Демографический сценарий	<code>sir_des_analysis_params.jl</code>
Вакцинация	<code>sir_des_analysis.jl</code> , <code>sir_des_analysis_params.jl</code>
SEIR-расширение	<code>sir_des_analysis_params.jl</code>

Таким образом, дополнительные задания были выполнены в рамках параметризованной и аналитической частей работы.

12 Итоговый вывод

В ходе лабораторной работы была реализована SIR-модель в дискретно-событийном подходе. Базовый запуск показал типичную эпидемическую динамику: рост числа инфицированных, достижение пика и последующее снижение. Параметризованный анализ подтвердил, что увеличение вероятности заражения и числа контактов усиливает эпидемию, а увеличение интенсивности выздоровления снижает пик инфекции.

Аналитическая часть показала, что DES-модель согласуется с детерминированной ODE-моделью по общей форме эпидемической волны, но отличается случайными колебаниями. Сценарий вакцинации снижает пик инфекции и итоговый масштаб эпидемии. Вариант с фиксированной длительностью болезни показал, что механизм выздоровления влияет на форму эпидемической кривой. Дополнительно были рассмотрены демографический сценарий и SEIR-расширение. Оценка производительности показала, что время выполнения увеличивается при росте размера популяции, что соответствует особенностям дискретно-событийного моделирования.

13 Список литературы

1. Ross S. M. *Simulation*. Academic Press, 2013.
2. Law A. M. *Simulation Modeling and Analysis*. McGraw-Hill, 2015.
3. Allen L. J. S. *An Introduction to Stochastic Processes with Applications to Biology*. CRC Press, 2010.
4. Kermack W. O., McKendrick A. G. A contribution to the mathematical theory of epidemics // *Proceedings of the Royal Society A*. 1927.
5. Julia Documentation. The Julia Programming Language. <https://docs.julialang.org/>
6. DifferentialEquations.jl Documentation. <https://docs.sciml.ai/DiffEqDocs/>
7. DrWatson.jl Documentation. <https://juliadynamics.github.io/DrWatson.jl/>

Список литературы

1. *Allen L. J. S.* An Introduction to Stochastic Epidemic Models. — Berlin, Heidelberg : Springer, 2008. — С. 81—130. — DOI: 10.1007/978-3-540-78911-6_3.
2. *Datseris G.* DrWatson.jl: The Perfect Sidekick to Your Scientific Inquiries. — 2026. — URL: <https://juliadynamics.github.io/DrWatson.jl/stable/> ; Julia package documentation.
3. Discrete-Event System Simulation / J. Banks [и др.]. — 5-е изд. — Upper Saddle River : Pearson, 2010.
4. *Ekre F.* Literate.jl Documentation. — 2026. — URL: <https://fredrikekre.github.io/Literate.jl/stable/> ; Julia package documentation.
5. *Hethcote H. W.* The Mathematics of Infectious Diseases. Т. 42. — Society for Industrial, Applied Mathematics, 2000. — С. 599—653. — DOI: 10.1137/S0036144500371907.
6. *JuliaData.* CSV.jl Documentation. — 2026. — URL: <https://csv.juliadata.org/stable/> ; Julia package documentation.
7. *JuliaData.* DataFrames.jl Documentation. — 2026. — URL: <https://dataframes.juliadata.org/stable/> ; Julia package documentation.
8. *JuliaLang.* The Julia Programming Language. — 2026. — URL: <https://julialang.org> ; Official documentation.

9. *Kermack W. O., McKendrick A. G. A Contribution to the Mathematical Theory of Epidemics // Proceedings of the Royal Society A. — 1927. — Т. 115, № 772. — С. 700—721. — DOI: 10.1098/rspa.1927.0118.*
10. *Law A. M. Simulation Modeling and Analysis. — 5-е изд. — New York : McGraw-Hill Education, 2015.*
11. *Plots.jl Contributors. Plots.jl Documentation. — 2026. — URL: <https://docs.juliaplots.org/stable/> ; Julia package documentation.*
12. *Posit Software, PBC. Quarto Documentation. — 2026. — URL: <https://quarto.org/docs/> ; Official documentation.*
13. *Ross S. M. Simulation. — 5-е изд. — Amsterdam : Academic Press, 2014.*
14. *SciML. DifferentialEquations.jl Documentation. — 2026. — URL: <https://docs.sciml.ai/DiffEqDocs/stable/> ; Julia package documentation.*
15. *Королькова А. В., Кулябов Д. С. Имитационное моделирование. Практикум. — Москва : Российский университет дружбы народов, 2025.*